

FILM MANAGEMENT SYSTEM REPORT

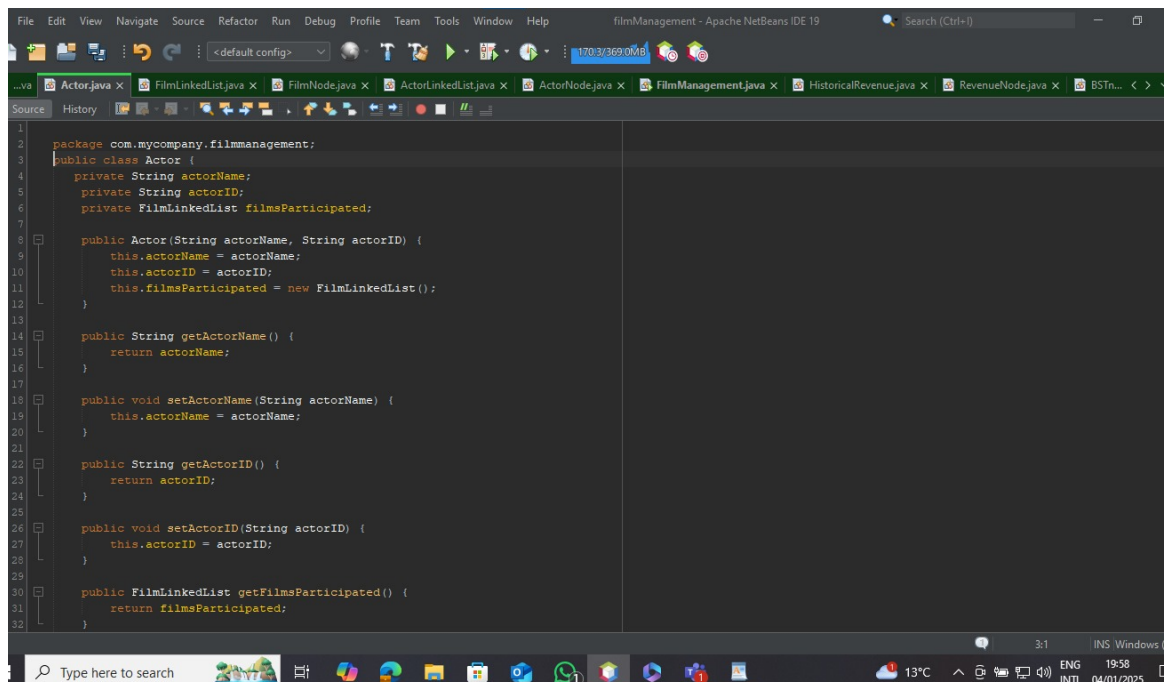
Prepared by : Buse Demirbaş 220315083 , Zehra Ismayilli 220315094 , Ece Kılıçaslan 220315038

This report outlines the design and implementation of the *Film Management System*, which aims to efficiently manage films, actors, production data, viewer feedback, and movie recommendations. The system demonstrates the practical application of key data structures such as Linked Lists, Queues, Stacks, Hash Tables, Heaps, and Binary Search Trees (BSTs) to address complex challenges in film management and data organization.

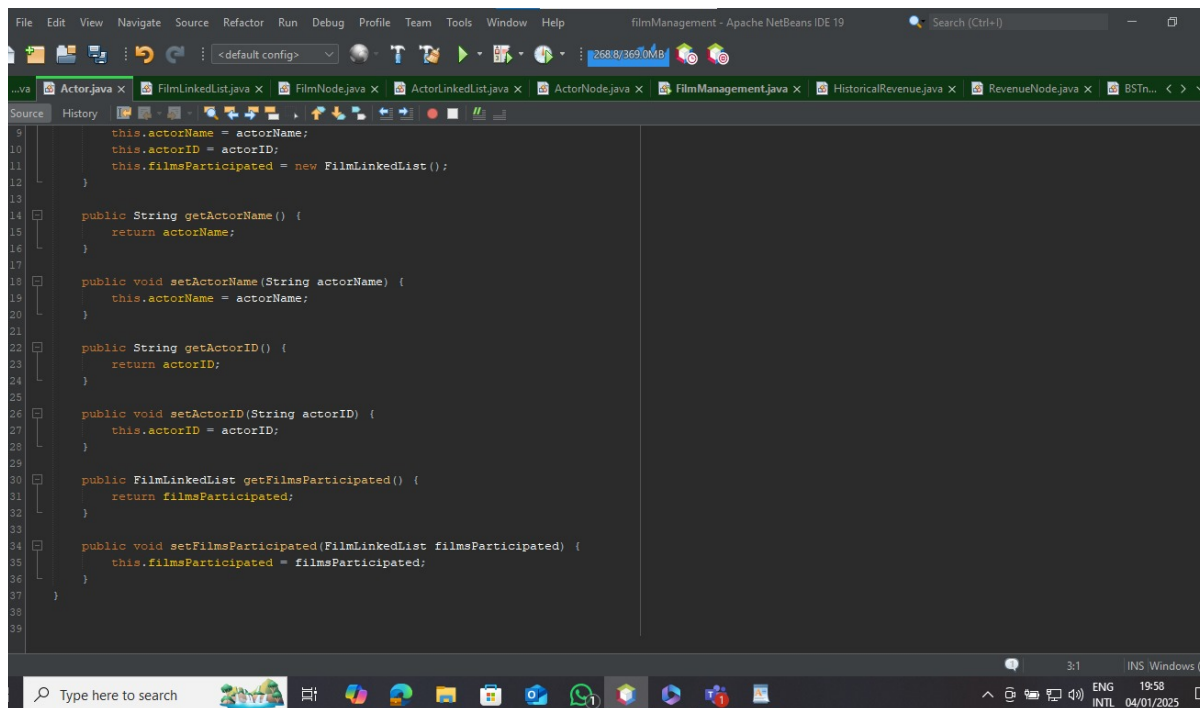
Now, let's define the classes to better understand their structure and functionality. **The Actor class** is designed to represent an actor in a film management system. This class stores essential information about the actor, such as their name, a unique identifier (actorID), and the list of films in which they have participated.

This class has three main parts: actorName for the actor's name, actorID for their unique ID, and filmsParticipated, which is a linked list (FilmLinkedList) that keeps track of the films the actor has been in. The class uses a constructor to set the name and ID of the actor and creates an empty list for their films.

The Actor class has some useful methods. The getActorName and setActorName methods are used to get or change the actor's name, while getActorID and setActorID do the same for the actor's ID. The getFilmsParticipated method shows the list of films, and the setFilmsParticipated method allows updating this list.



```
1 package com.mycompany.filmmanagement;
2
3 public class Actor {
4     private String actorName;
5     private String actorID;
6     private FilmLinkedList filmsParticipated;
7
8     public Actor(String actorName, String actorID) {
9         this.actorName = actorName;
10        this.actorID = actorID;
11        this.filmsParticipated = new FilmLinkedList();
12    }
13
14    public String getActorName() {
15        return actorName;
16    }
17
18    public void setActorName(String actorName) {
19        this.actorName = actorName;
20    }
21
22    public String getActorID() {
23        return actorID;
24    }
25
26    public void setActorID(String actorID) {
27        this.actorID = actorID;
28    }
29
30    public FilmLinkedList getFilmsParticipated() {
31        return filmsParticipated;
32    }
33 }
```



```
9      this.actorName = actorName;
10     this.actorID = actorID;
11     this.filmsParticipated = new FilmLinkedList();
12 }
13
14 public String getActorName() {
15     return actorName;
16 }
17
18 public void setActorName(String actorName) {
19     this.actorName = actorName;
20 }
21
22 public String getActorID() {
23     return actorID;
24 }
25
26 public void setActorID(String actorID) {
27     this.actorID = actorID;
28 }
29
30 public FilmLinkedList getFilmsParticipated() {
31     return filmsParticipated;
32 }
33
34 public void setFilmsParticipated(FilmLinkedList filmsParticipated) {
35     this.filmsParticipated = filmsParticipated;
36 }
37 }
38
39 }
```

The Actor Class

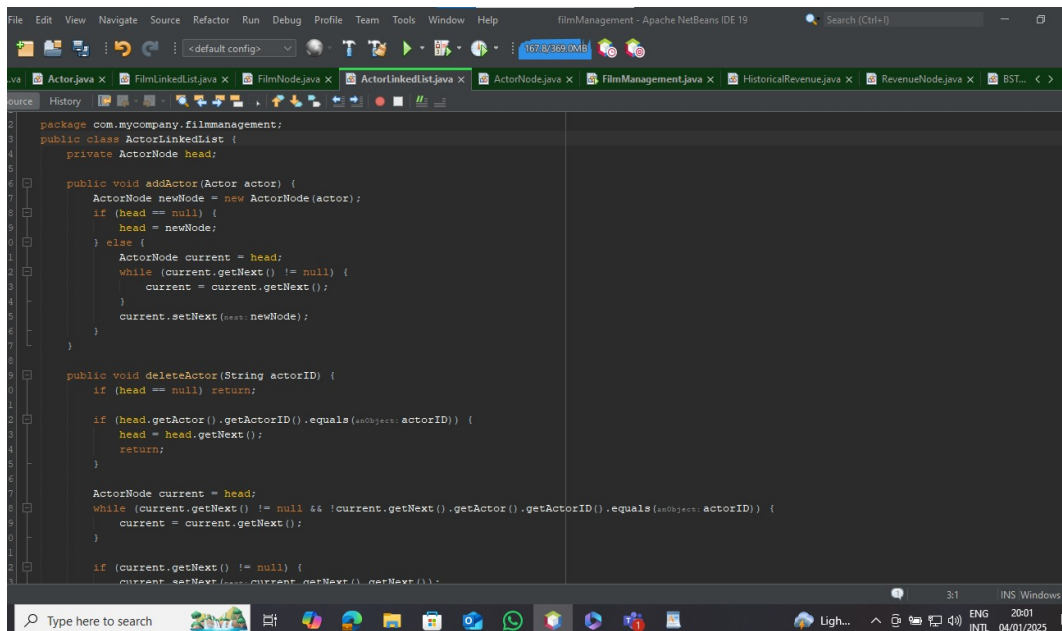
Now, let's explain **the ActorLinkedList class** and its functionality. The ActorLinkedList class is a custom implementation of a linked list designed to manage actors in the film management system. This class allows adding, deleting, and printing actors efficiently using a singly linked list structure.

The addActor method is used to add a new actor to the list. If the list is empty, the new actor becomes the first node, also called the head. If the list already contains actors, the method traverses to the end of the list and adds the new actor there. This ensures that new actors are always appended to the list.

The deleteActor method is responsible for removing an actor from the list based on their unique ID, called actorID. If the actor to be deleted is the first one in the list, the head pointer is moved to the next node, effectively removing the first actor. If the actor is somewhere in the middle or at the end of the list, the method finds the actor and updates the links to skip over the actor being deleted.

The printActors method is used to display all actors in the list. If the list is empty, it prints a message saying, "No actors in the list." Otherwise, it goes through each node in the list and prints the name and ID of every actor. This is useful for viewing the current list of actors in the system.

In summary, the ActorLinkedList class is a simple but powerful way to manage actors. It allows you to dynamically add new actors, remove actors as needed, and view all the actors in the system. This makes it an important part of managing actor-related information in the film management system.



```
package com.mycompany.filmmanagement;

public class ActorLinkedList {
    private ActorNode head;

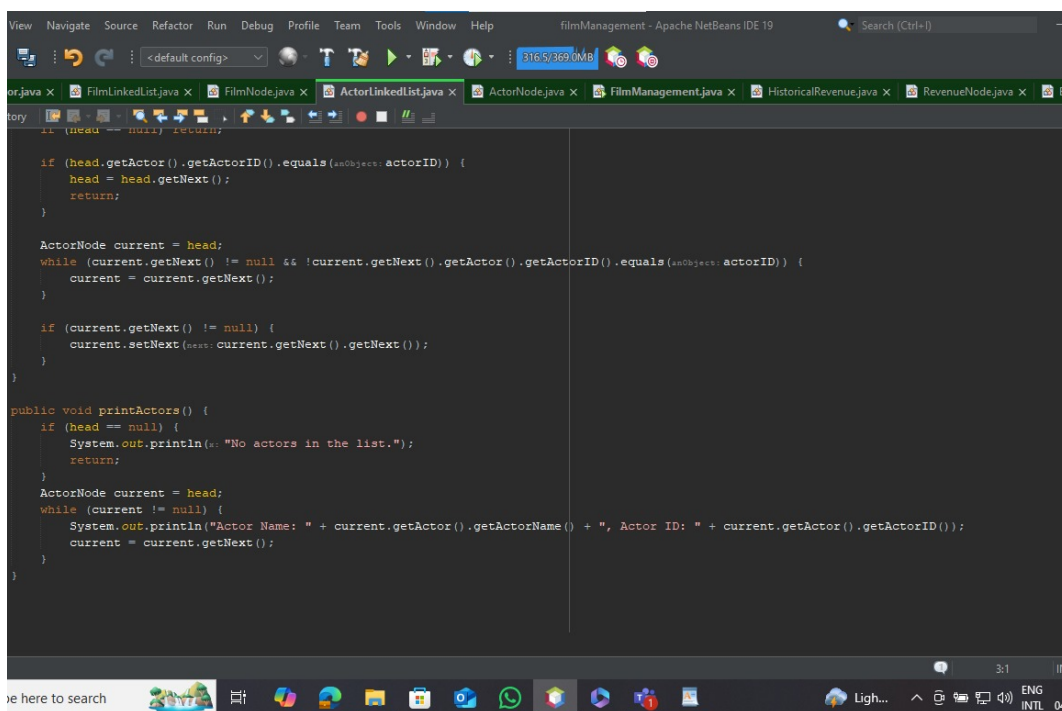
    public void addActor(Actor actor) {
        ActorNode newNode = new ActorNode(actor);
        if (head == null) {
            head = newNode;
        } else {
            ActorNode current = head;
            while (current.getNext() != null) {
                current = current.getNext();
            }
            current.setNext(newNode);
        }
    }

    public void deleteActor(String actorID) {
        if (head == null) return;

        if (head.getActor().getActorID().equals(actorID)) {
            head = head.getNext();
            return;
        }

        ActorNode current = head;
        while (current.getNext() != null && !current.getNext().getActor().getActorID().equals(actorID)) {
            current = current.getNext();
        }

        if (current.getNext() != null) {
            current.setNext(current.getNext().getNext());
        }
    }
}
```



```
if (head == null) return;

if (head.getActor().getActorID().equals(actorID)) {
    head = head.getNext();
    return;
}

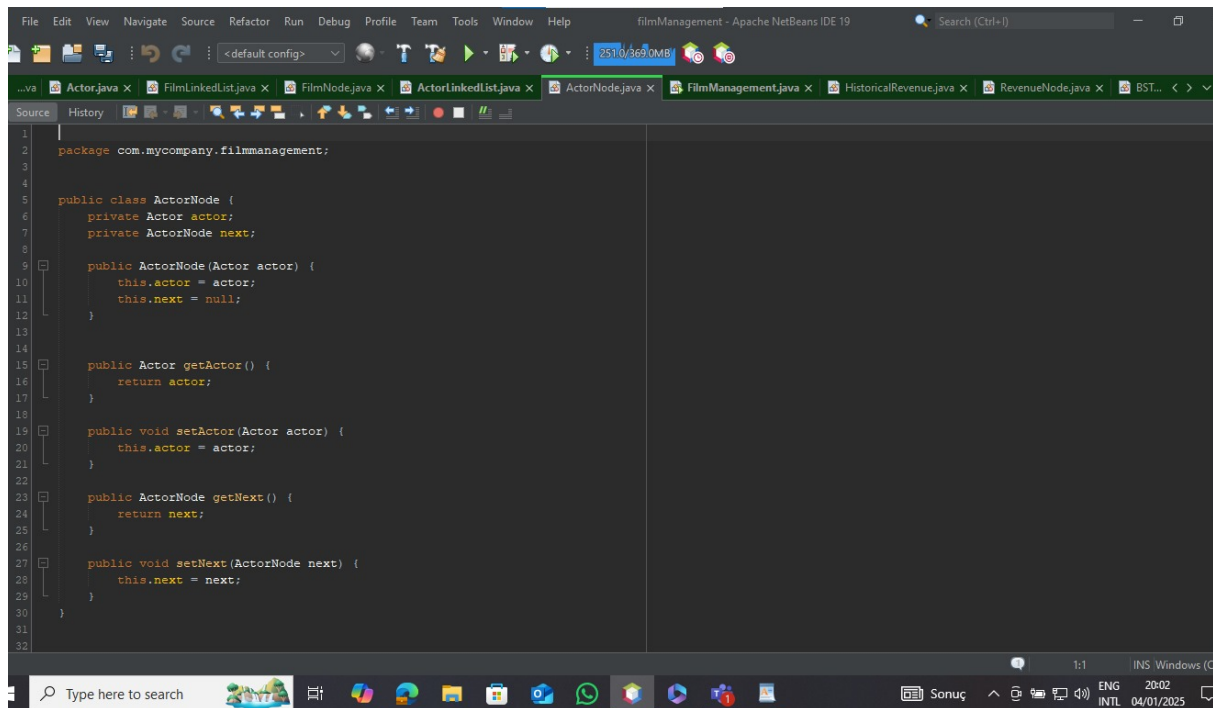
ActorNode current = head;
while (current.getNext() != null && !current.getNext().getActor().getActorID().equals(actorID)) {
    current = current.getNext();
}

if (current.getNext() != null) {
    current.setNext(current.getNext().getNext());
}

public void printActors() {
    if (head == null) {
        System.out.println("No actors in the list.");
        return;
    }
    ActorNode current = head;
    while (current != null) {
        System.out.println("Actor Name: " + current.getActor().getActorName() + ", Actor ID: " + current.getActor().getActorID());
        current = current.getNext();
    }
}
```

The ActorLinkedList Class

The ActorNode class is a part of the linked list structure for managing actors. Each node in the list stores an Actor object, which contains information about an actor, and a reference to the next node in the list. The getActor and setActor methods allow accessing or updating the actor stored in the node, while the getNext and setNext methods manage the connection to the next node. The ActorNode helps build the linked list by connecting multiple nodes together to store a dynamic list of actors.



```
1 package com.mycompany.filmmanagement;
2
3
4
5 public class ActorNode {
6     private Actor actor;
7     private ActorNode next;
8
9     public ActorNode(Actor actor) {
10         this.actor = actor;
11         this.next = null;
12     }
13
14
15     public Actor getActor() {
16         return actor;
17     }
18
19     public void setActor(Actor actor) {
20         this.actor = actor;
21     }
22
23     public ActorNode getNext() {
24         return next;
25     }
26
27     public void setNext(ActorNode next) {
28         this.next = next;
29     }
30 }
31
32
```

The ActorNode Class

Then , **The Film class** is an essential part of the film management system. It is responsible for storing and managing all details related to a film, such as its name, ID, genre, release year, revenue, reviews, IMDb ratings, and popularity. This class provides various methods to access, update, and display these details. Below is an explanation of each method in simple terms:

Constructor: Film(String filmName, String filmID, String genre, int releaseYear)

The constructor is the method used to create a new film object. It sets the basic details of the film, including its name, ID, genre, and release year. It also initializes default values like total revenue (set to 0.0), average rating (0.0), and popularity score (0.0). Additionally, it creates a **FilmScreening** object to handle IMDb ratings for the film.

Getter and Setter Methods (e.g., getFilmName, setFilmName)

These methods allow you to retrieve or update the film's details. For instance, `getFilmName` returns the name of the film, while `setFilmName` allows changing it. Similar methods exist for attributes like ID, genre, and release year.

addReview(String reviewerName, double rating, String comment)

This method lets users add a review for the film. The review includes the reviewer's name, a rating (e.g., 8.0), and a comment (e.g., "Great film!"). These details are stored in the reviews list.

deleteReview(String reviewerName)

This method removes a review based on the reviewer's name. If a review with the given name exists, it is deleted from the reviews list.

printReviews()

This method displays all reviews for the film. It shows the reviewer's name, their rating, and any comments they provided.

addRevenue(double viewership, double ticketPrice)

This method records revenue data for the film. It calculates the revenue by multiplying the number of viewers (viewership) by the ticket price and saves the information in the historicalRevenue object.

updateTotalRevenue()

This method updates the film's total revenue by summing up all revenue records stored in historicalRevenue.

printRevenue()

This method prints the film's total revenue along with its name and ID. It provides a quick summary of how much money the film has earned.

printRevenueHistory()

This method displays the complete revenue history for the film. It includes details such as viewership numbers, ticket prices, and the total revenue for each entry.

addImdb(double imdb, String filmName)

This method adds an IMDb rating to the film. It updates the IMDb ratings list and recalculates the film's average IMDb rating and popularity score.

printAllImdb()

This method prints all IMDb ratings for the film. If no ratings are available, it displays a message saying "No IMDb ratings available."

calculateAverageRating()

This method calculates the average IMDb rating for the film by adding all ratings and dividing by the total number of ratings.

calculatePopularityScore()

This method calculates the film's popularity score by combining its total revenue and average IMDb rating. A higher score means the film is more popular.

printBestFilm()

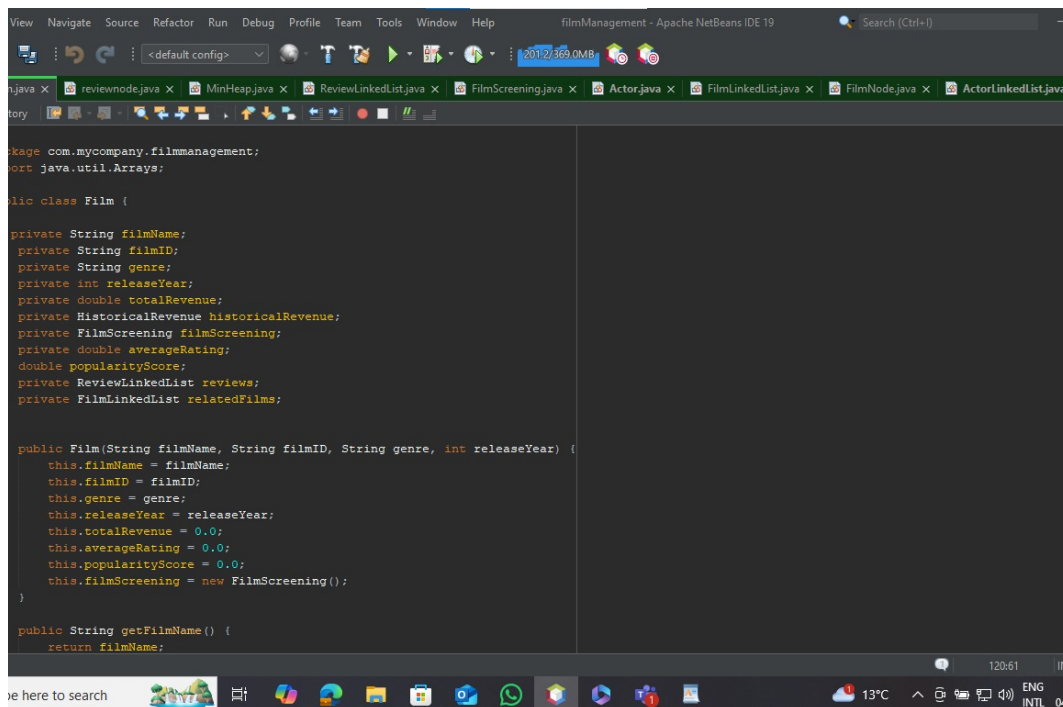
This method identifies and displays the best film based on IMDb ratings. It finds the highest rating and shows the film's name and its score.

displayPopularityRankings(Film[] films)

This method sorts all films by their popularity scores and displays them from most popular to least popular.

toString()

This method converts the film's details into a string format, providing a quick summary that includes the film's name, ID, genre, and release year.



The screenshot shows the Apache NetBeans IDE with the 'Film' class defined in the 'com.mycompany.filmmanagement' package. The class includes private attributes for film details and public methods for initialization and retrieval. The memory usage is 201.2/369.0MB.

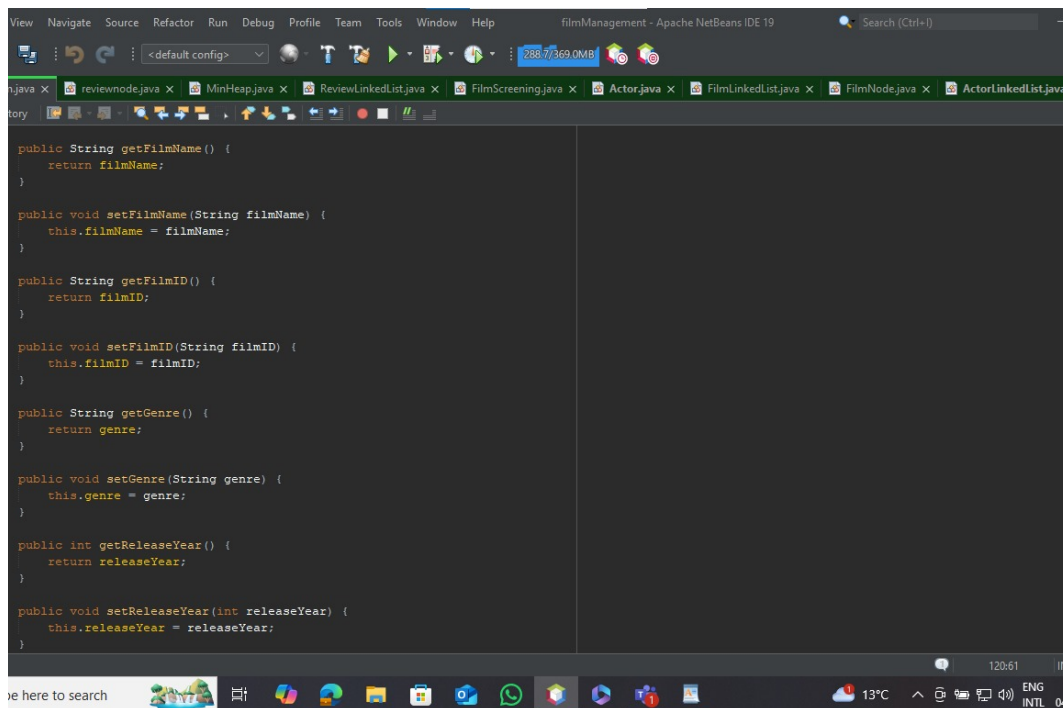
```
package com.mycompany.filmmanagement;
import java.util.Arrays;

public class Film {

    private String filmName;
    private String filmID;
    private String genre;
    private int releaseYear;
    private double totalRevenue;
    private HistoricalRevenue historicalRevenue;
    private FilmScreening filmScreening;
    private double averageRating;
    double popularityScore;
    private ReviewLinkedList reviews;
    private FilmLinkedList relatedFilms;

    public Film(String filmName, String filmID, String genre, int releaseYear) {
        this.filmName = filmName;
        this.filmID = filmID;
        this.genre = genre;
        this.releaseYear = releaseYear;
        this.totalRevenue = 0.0;
        this.averageRating = 0.0;
        this.popularityScore = 0.0;
        this.filmScreening = new FilmScreening();
    }

    public String getFilmName() {
        return filmName;
    }
}
```



The screenshot shows the continuation of the 'Film' class with getter and setter methods for filmName, filmID, genre, and releaseYear. The memory usage is 288.7/369.0MB.

```
    public String getFilmName() {
        return filmName;
    }

    public void setFilmName(String filmName) {
        this.filmName = filmName;
    }

    public String getFilmID() {
        return filmID;
    }

    public void setFilmID(String filmID) {
        this.filmID = filmID;
    }

    public String getGenre() {
        return genre;
    }

    public void setGenre(String genre) {
        this.genre = genre;
    }

    public int getReleaseYear() {
        return releaseYear;
    }

    public void setReleaseYear(int releaseYear) {
        this.releaseYear = releaseYear;
    }
}
```



```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help FilmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
248.1/369.0MB

reviewnode.java X MinHeap.java X ReviewLinkedList.java X FilmScreening.java X Actor.java X FilmLinkedList.java X FilmNode.java X ActorLinkedList.java
History

public double getTotalRevenue() {
    return totalRevenue;
}

public void setTotalRevenue(double totalRevenue) {
    this.totalRevenue = totalRevenue;
}

public HistoricalRevenue getHistoricalRevenue() {
    return historicalRevenue;
}

public void setHistoricalRevenue(HistoricalRevenue historicalRevenue) {
    this.historicalRevenue = historicalRevenue;
}

public FilmScreening getFilmScreening() {
    return filmScreening;
}

public void setFilmScreening(FilmScreening filmScreening) {
    this.filmScreening = filmScreening;
}

public double getPopularityScore() {
    return popularityScore;
}

public void setPopularityScore(double popularityScore) {
    this.popularityScore = popularityScore;
}

47:31 IN
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help FilmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
179.0/369.0MB

reviewnode.java X MinHeap.java X ReviewLinkedList.java X FilmScreening.java X Actor.java X FilmLinkedList.java X FilmNode.java X ActorLinkedList.java
History

    this.popularityScore = popularityScore;
}

public double getAverageRating() {
    return averageRating;
}

public void setAverageRating(double averageRating) {
    this.averageRating = averageRating;
}

public FilmLinkedList getRelatedFilms() {
    return relatedFilms;
}

public void addReview(String reviewerName, double rating, String comment) {
    Review review = new Review(reviewerName, rating, comment);
    reviews.addReview(review);
}

public void deleteReview(String reviewerName) {
    reviews.deleteReview(reviewerName);
}

public void printReviews() {
    System.out.println("Reviews for film: " + filmName);
    reviews.printReviews();
}

public void addRevenue(double viewership, double ticketPrice) {
    // ...
}

47:31 IN
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help FilmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
290.1/369.0MB

reviewnode.java X MinHeap.java X ReviewLinkedList.java X FilmScreening.java X Actor.java X FilmLinkedList.java X FilmNode.java X ActorLinkedList.java
History

public void addRevenue(double viewership, double ticketPrice) {
    if (historicalRevenue == null) {
        historicalRevenue = new HistoricalRevenue();
    }
    historicalRevenue.push(viewership, ticketPrice);
    updateTotalRevenue();
}

public void updateTotalRevenue() {
    double total = 0;
    RevenueNode current = historicalRevenue.top;
    while (current != null) {
        total += current.totalRevenue;
        current = current.next;
    }
    this.totalRevenue = total;
}

public void printRevenue() {
    System.out.println("Film: " + filmName + " (ID: " + filmID + ")");
    System.out.println("Total Revenue: " + totalRevenue);
}

public void printRevenueHistory() {
    System.out.println("Revenue History for " + filmName + " (ID: " + filmID + ")");
    RevenueNode current = historicalRevenue.top;
    while (current != null) {
        System.out.println("Viewership: " + current.viewership + ", Ticket Price: " + current.ticketPrice + ", Total Revenue: " + current.totalRevenue);
        current = current.next;
    }
}

47:31 IN
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
<default config> 219.3/369.0MB
reviewnode.java X MinHeap.java X ReviewLinkedList.java X FilmScreening.java X Actor.java X FilmLinkedList.java X FilmNode.java X ActorLinkedList.java
}

public static void displayPopularityRankings(Film[] films) {
    System.out.println("Displaying Popularity Rankings:");

    Arrays.sort(films, (f1, f2) -> Double.compare(f2.getPopularityScore(), f1.getPopularityScore()));

    for (Film film : films) {
        if (film != null) {
            System.out.println(film.getFilmName() + " - Popularity Score: " + film.getPopularityScore());
        }
    }
}

public void addImdb(double imdb, String filmName) {
    filmScreening.enqueue(imdb, filmName);
    calculateAverageRating();
    calculatePopularityScore();
}

public void printAllImdb() {
    if (filmScreening.isEmpty()) {
        System.out.println("No IMDb ratings available for this film.");
        return;
    }
    System.out.println(filmName + " IMDb ratings:");
    FilmScreening tempScreening = new FilmScreening();
    while (!filmScreening.isEmpty()) {
        ImdbNode node = filmScreening.dequeue();
        System.out.println("IMDb: " + node.imdb);
        tempScreening.enqueue(node.imdb, node.filmName);
    }
}
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
<default config> 161.0/369.0MB
reviewnode.java X MinHeap.java X ReviewLinkedList.java X FilmScreening.java X Actor.java X FilmLinkedList.java X FilmNode.java X ActorLinkedList.java
}

while (!tempScreening.isEmpty()) {
    ImdbNode node = tempScreening.dequeue();
    filmScreening.enqueue(node.imdb, node.filmName);
}

public void calculateAverageRating() {
    if (filmScreening.isEmpty()) {
        averageRating = 0.0;
        return;
    }

    double totalRating = 0.0;
    int count = 0;
    FilmScreening tempScreening = new FilmScreening();

    while (!filmScreening.isEmpty()) {
        ImdbNode node = filmScreening.dequeue();
        totalRating += node.imdb;
        count++;
        tempScreening.enqueue(node.imdb, node.filmName);
    }

    while (!tempScreening.isEmpty()) {
        ImdbNode node = tempScreening.dequeue();
        filmScreening.enqueue(node.imdb, node.filmName);
    }

    this.averageRating = totalRating / count;
}
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
<default config> 161.0/369.0MB
reviewnode.java X MinHeap.java X ReviewLinkedList.java X FilmScreening.java X Actor.java X FilmLinkedList.java X FilmNode.java X ActorLinkedList.java
}

while (!tempScreening.isEmpty()) {
    ImdbNode node = tempScreening.dequeue();
    filmScreening.enqueue(node.imdb, node.filmName);
}

public void calculateAverageRating() {
    if (filmScreening.isEmpty()) {
        averageRating = 0.0;
        return;
    }

    double totalRating = 0.0;
    int count = 0;
    FilmScreening tempScreening = new FilmScreening();

    while (!filmScreening.isEmpty()) {
        ImdbNode node = filmScreening.dequeue();
        totalRating += node.imdb;
        count++;
        tempScreening.enqueue(node.imdb, node.filmName);
    }

    while (!tempScreening.isEmpty()) {
        ImdbNode node = tempScreening.dequeue();
        filmScreening.enqueue(node.imdb, node.filmName);
    }

    this.averageRating = totalRating / count;
}
```



```

temp.enqueue(imdb: node.imdb, fileName: node.fileName);
}
while (!temp.isEmpty()) {
    ImdbNode node = temp.dequeue();
    filmScreening.enqueue(imdb: node.imdb, fileName: node.fileName);
}
System.out.println("The best film is: " + bestFilmName + " with IMDB: " + maxImdb);
}

public String toString() {
    return "Film Name: " + this.fileName + ", ID: " + this.filmID + ", Genre: " + this.genre + ", Release Year: " + this.releaseYear;
}

```

The Film Class

The FilmLinkedList class provides a structure to manage a list of films using a linked list data structure..The addFilm(Film film) method allows adding a new film to the list by either setting it as the head if the list is empty or appending it to the end of the list. The deleteFilm(String fileName) method removes a film with the specified name from the list, ensuring the head is updated if the first film matches the criteria. To determine the number of films in the list, the getSize() method traverses the list and returns the total count of nodes. The fillArray(Film[] films) method copies all film objects from the linked list into an array, enabling easier access and manipulation of the film collection.

```

package com.mycompany.filmmanagement;

public class FilmLinkedList {
    private FilmNode head;

    public void addFilm(Film film) {
        FilmNode newNode = new FilmNode(film);
        if (head == null) {
            head = newNode;
        } else {
            FilmNode current = head;
            while (current.getNext() != null) {
                current = current.getNext();
            }
            current.setNext(newNode);
        }
    }

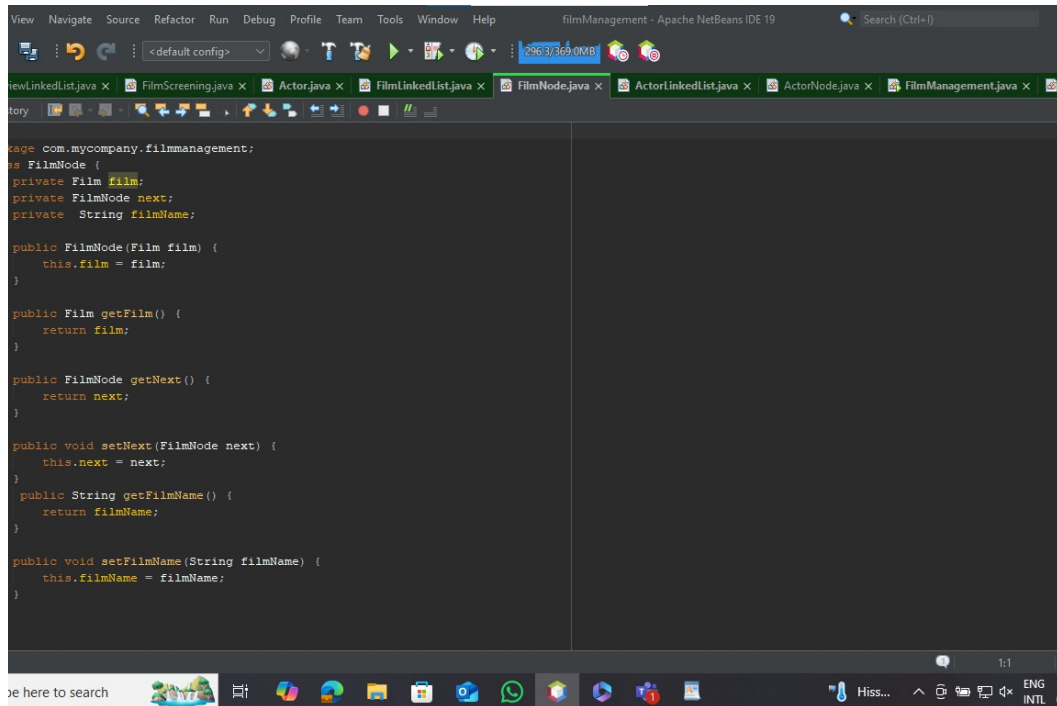
    public void deleteFilm(String fileName) {
        if (head == null) return;
        if (head.getFilm().getFileName().equals(fileName)) {
            head = head.getNext();
            return;
        }
        FilmNode current = head;
        while (current.getNext() != null && !current.getNext().getFilm().getFileName().equals(fileName)) {
            current = current.getNext();
        }
        if (current.getNext() != null) {
            current.setNext(current.getNext().getNext());
        }
    }

    public int getSize() {
        int size = 0;
    }
}

```

The FilmLinkedList Class

The *FilmNode* class represents a single node in the linked list for managing films. Each node stores a *Film* object, the name of the film (*filmName*), and a reference to the next node (*next*). The class includes a constructor to initialize the node and getter and setter methods to access and update its data. It is a basic building block for the linked list structure.

A screenshot of the Apache NetBeans IDE showing the `FilmNode.java` file. The code defines a `FilmNode` class with private attributes `film`, `next`, and `filmName`. It includes a constructor `FilmNode(Film film)`, a `getFilm()` method, a `getNext()` method, a `setNext(FilmNode next)` method, a `getFilmName()` method, and a `setFilmName(String filmName)` method. The IDE interface shows various tabs at the top, including `FilmScreening.java`, `Actor.java`, `FilmLinkedList.java`, `FilmNode.java`, `ActorLinkedList.java`, `ActorNode.java`, and `FilmManagement.java`. The status bar at the bottom indicates the file is at line 1, column 1.

```
package com.mycompany.filmmanagement;

import java.util.*;

public class FilmNode {
    private Film film;
    private FilmNode next;
    private String filmName;

    public FilmNode(Film film) {
        this.film = film;
    }

    public Film getFilm() {
        return film;
    }

    public FilmNode getNext() {
        return next;
    }

    public void setNext(FilmNode next) {
        this.next = next;
    }

    public String getFilmName() {
        return filmName;
    }

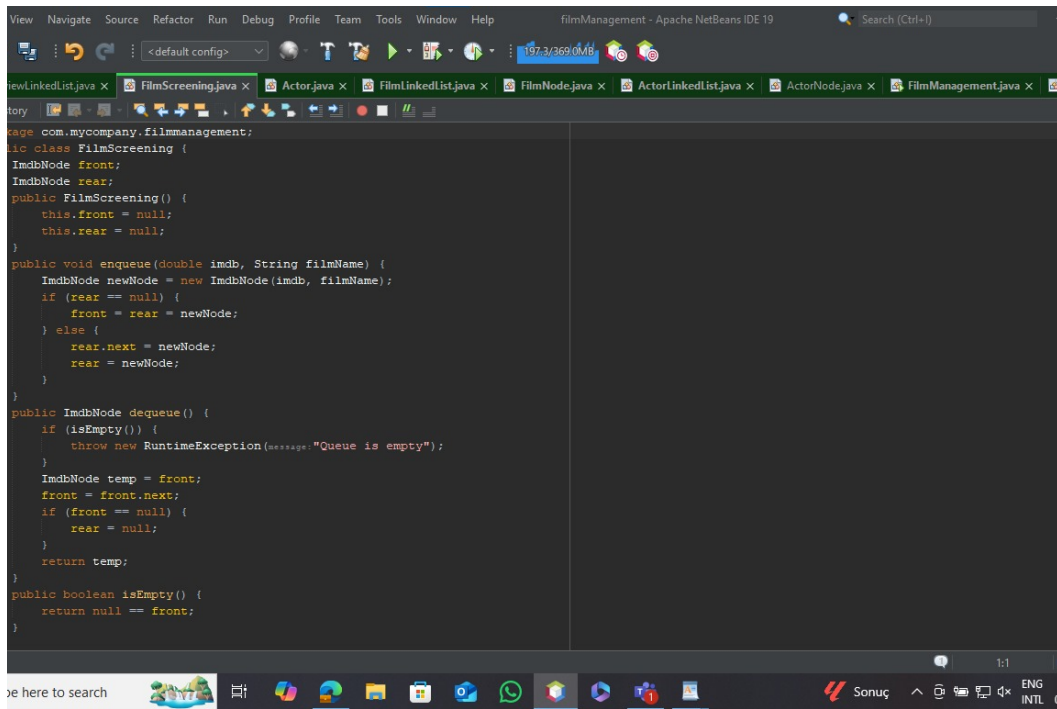
    public void setFilmName(String filmName) {
        this.filmName = filmName;
    }
}
```

The *filmNode* Class

The *FilmScreening* class is a queue used to manage films. It has two main attributes: *front*, which points to the first film in the queue, and *rear*, which points to the last film.

The constructor initializes the queue as empty by setting both *front* and *rear* to null. The *enqueue* method adds a new film to the end of the queue. If the queue is empty, the new film becomes both the *front* and *rear*. Otherwise, it is linked to the current *rear*, and the *rear* is updated.

The *dequeue* method removes the film at the front of the queue. If the queue is empty, it throws an exception. The *isEmpty* method checks if the queue is empty by seeing if *front* is null. This class is useful for processing films in the order they were added.



The FilmScreening Class

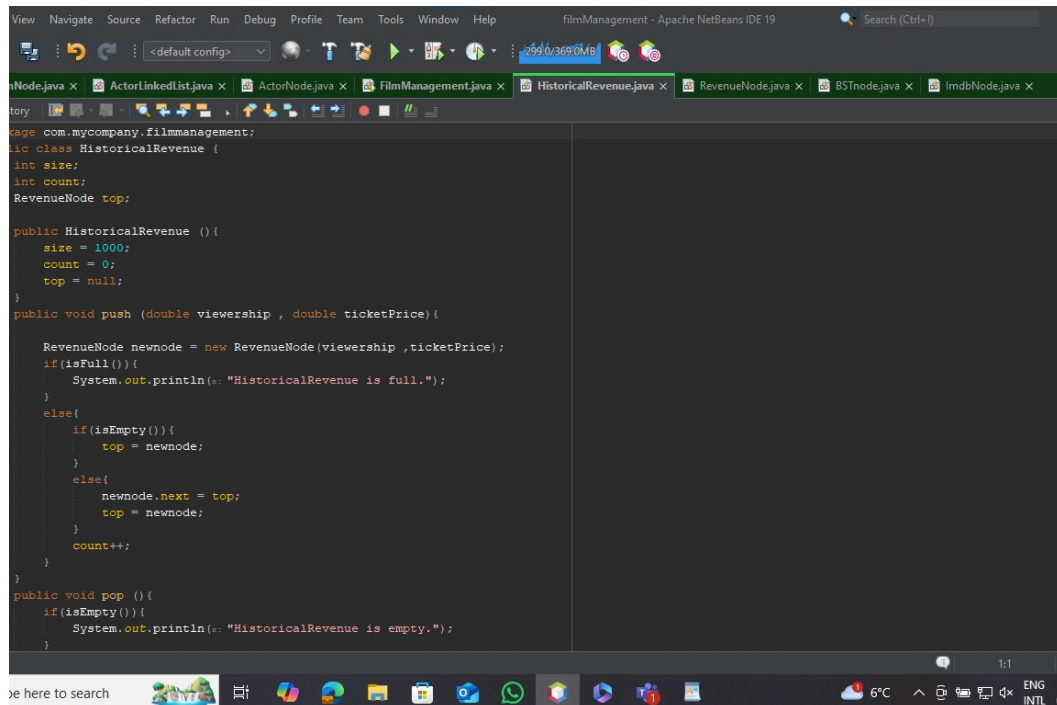
The HistoricalRevenue class represents a stack structure for managing revenue data, such as viewership and ticket prices. It uses a RevenueNode object to store each data entry and tracks the top of the stack with the top attribute.

The class has three main attributes: size (the maximum size of the stack, set to 1000), count (the current number of elements in the stack), and top (the reference to the top node of the stack).

The push(double viewership, double ticketPrice) method adds a new revenue record to the top of the stack. If the stack is full, it prints a message indicating that no more elements can be added. Otherwise, the new node is linked to the current top, and top is updated to this new node, incrementing the count.

The pop() method removes the top element from the stack. If the stack is empty, it prints a message indicating this. Otherwise, the top pointer is updated to the next node, and the count is decremented.

The isFull() method checks if the stack has reached its maximum size, and the isEmpty() method checks if the stack is empty. This class is ideal for managing historical revenue records in a last-in, first-out (LIFO) manner.



```
package com.myccompany.filmmanagement;

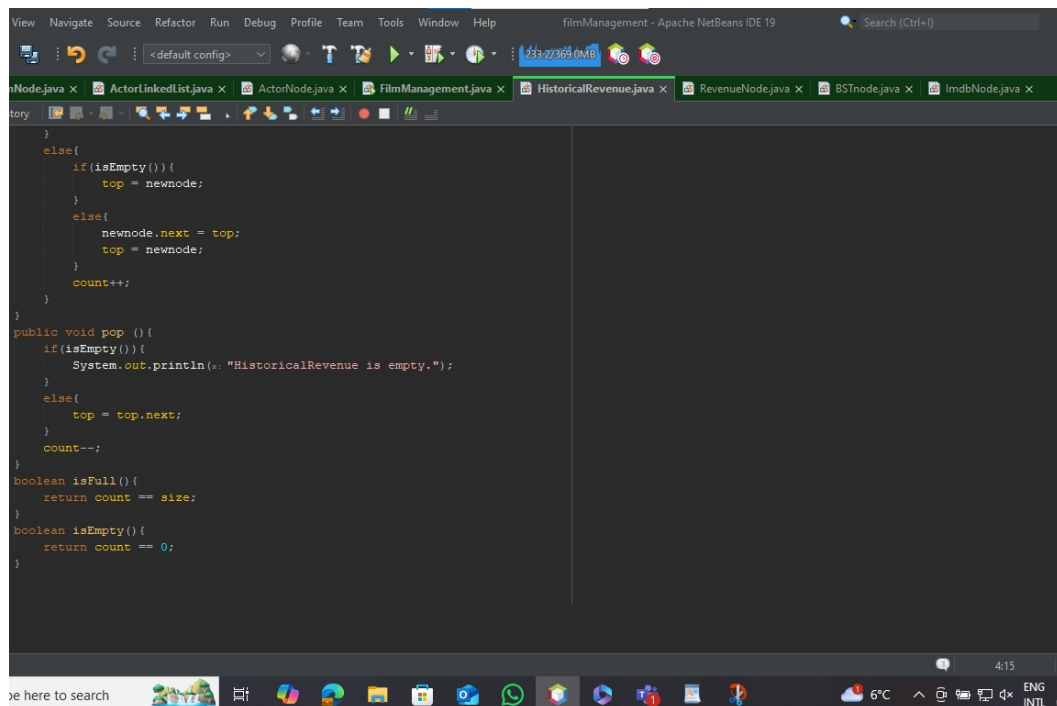
public class HistoricalRevenue {
    int size;
    int count;
    RevenueNode top;

    public HistoricalRevenue () {
        size = 1000;
        count = 0;
        top = null;
    }

    public void push (double viewership , double ticketPrice){

        RevenueNode newnode = new RevenueNode(viewership ,ticketPrice);
        if(isFull()){
            System.out.println("HistoricalRevenue is full.");
        }
        else{
            if(isEmpty()){
                top = newnode;
            }
            else{
                newnode.next = top;
                top = newnode;
            }
            count++;
        }
    }

    public void pop () {
        if(isEmpty()){
            System.out.println("HistoricalRevenue is empty.");
        }
    }
}
```



```
        }
        else{
            if(isEmpty()){
                top = newnode;
            }
            else{
                newnode.next = top;
                top = newnode;
            }
            count++;
        }
    }

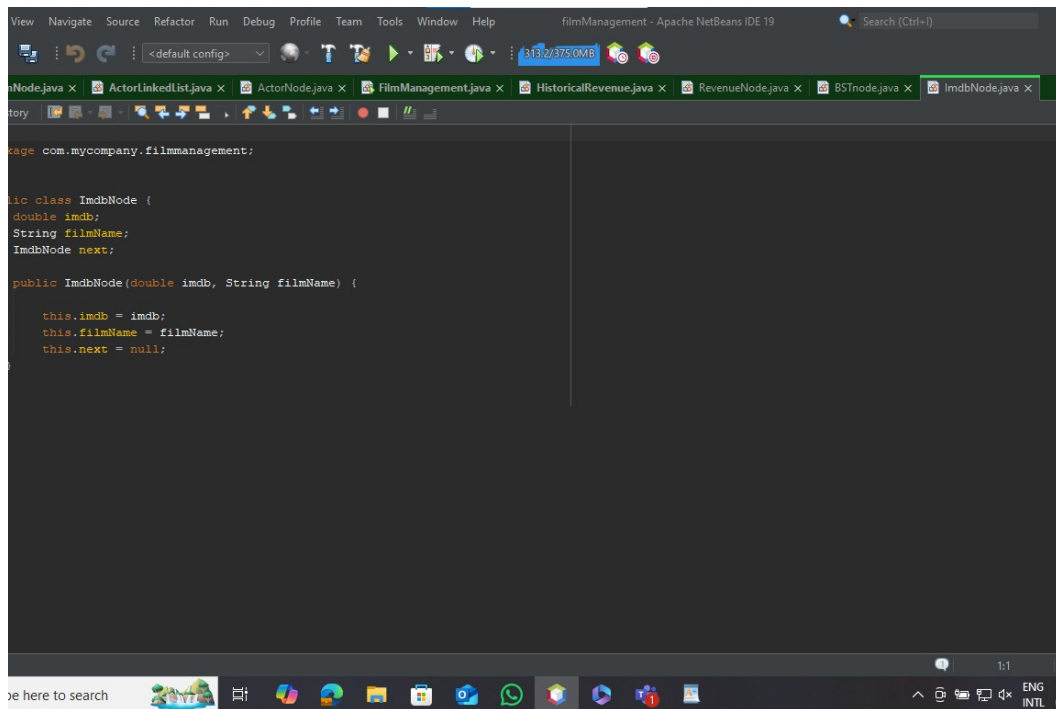
    public void pop () {
        if(isEmpty()){
            System.out.println("HistoricalRevenue is empty.");
        }
        else{
            top = top.next;
        }
        count--;
    }

    boolean isFull() {
        return count == size;
    }

    boolean isEmpty(){
        return count == 0;
    }
}
```

The HistoricalRevenue Class

The *ImdbNode* class represents a single node in a linked data structure used to manage films. Each node contains three attributes: imdb, which stores the IMDb rating of the film as a double; filmName, which holds the name of the film as a string; and next, a reference to the next node in the structure. The class includes a constructor that initializes the IMDb rating and film name while setting the next reference to null. This class serves as a fundamental element for organizing and managing film data in structures like linked lists or queues.



The ImdbNode Class

The MinHeap class implements a min-heap data structure to organize films based on their popularity scores, ensuring the smallest value (lowest popularity score) is always at the top. The class has three main attributes: heap, an array to store films; maxSize, the maximum number of films the heap can hold; and currentSize, the current number of films in the heap.

The constructor initializes the heap array and sets the maxSize and currentSize. The insert(Film film) method adds a film to the heap. If the heap is full, a message is printed, and the insertion is aborted. After inserting a new film, the method ensures the heap property is maintained by comparing and swapping nodes as necessary.

The extractMin() method removes and returns the film with the lowest popularity score (the root). If the heap is empty, it prints an error message. After removing the root, the heap is reorganized using the minHeapify(int i) method to maintain the min-heap property. Utility methods like parent(int i), leftChild(int i), and rightChild(int i) help navigate the heap structure. The printHeap() method prints all films in the heap, making it easy to view the current state. This class is ideal for efficiently managing and retrieving films based on their popularity scores.


```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
Heap.java X ReviewLinkedList.java X FilmScreening.java X Actor.java X FilmLinkedList.java X FilmNode.java X ActorLinkedList.java X ActorNode.java X FilmMa
tory
package com.mycompany.filmmanagement;
public class MinHeap {
    private final int maxSize;
    private final Film[] heap;
    private int currentSize;
    public MinHeap(int maxSize) {
        this.heap = new Film[maxSize];
        this.currentSize = -1;
        this.maxSize = maxSize;
    }
    public int parent(int i) {
        return (i - 1) / 2;
    }
    public int leftChild(int i) {
        return 2 * i + 1;
    }
    public int rightChild(int i) {
        return 2 * i + 2;
    }
    public void insert(Film film) {
        if (currentSize == maxSize - 1) {
            System.out.println("The heap is full. Cannot insert");
            return;
        }
        currentSize++;
        heap[currentSize] = film;
        int currentIndex = currentSize;
        while (currentIndex > 0 && heap[parent(i, currentIndex)].popularityScore > heap[currentIndex].popularityScore) {
            Film temp = heap[currentIndex];
            heap[currentIndex] = heap[parent(i, currentIndex)];
            heap[parent(i, currentIndex)] = temp;
            currentIndex = parent(i, currentIndex);
        }
    }
}
```

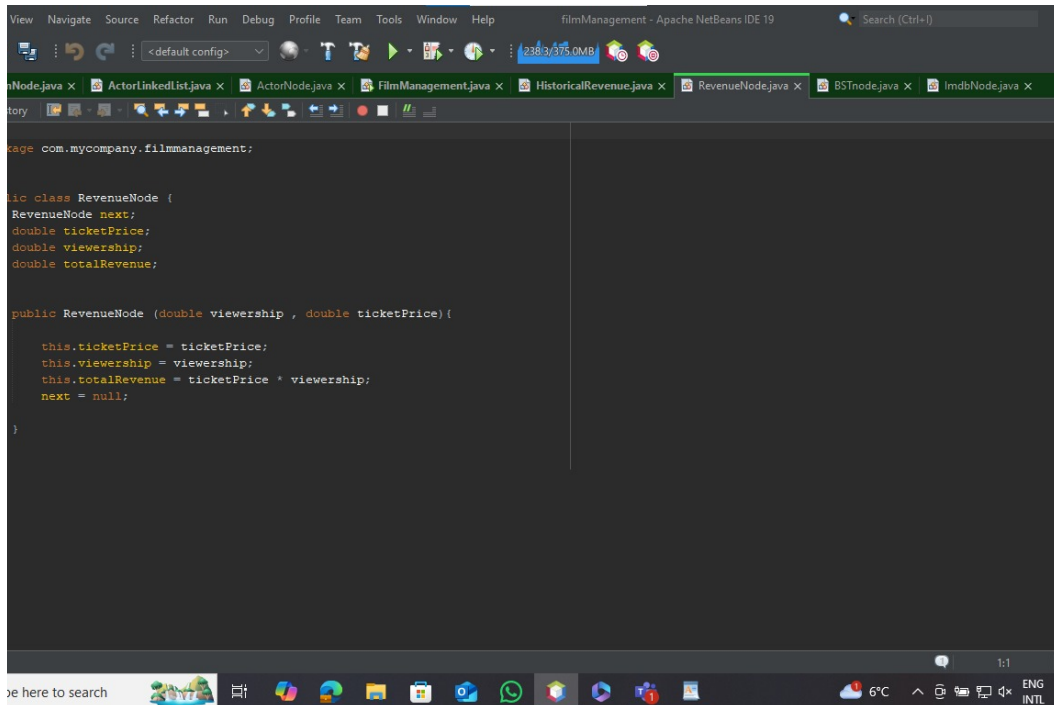
```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
Heap.java X ReviewLinkedList.java X FilmScreening.java X Actor.java X FilmLinkedList.java X FilmNode.java X ActorLinkedList.java X ActorNode.java X FilmMa
tory
while (currentIndex > 0 && heap[parent(i, currentIndex)].popularityScore > heap[currentIndex].popularityScore) {
    Film temp = heap[currentIndex];
    heap[currentIndex] = heap[parent(i, currentIndex)];
    heap[parent(i, currentIndex)] = temp;
    currentIndex = parent(i, currentIndex);
}

public Film extractMin() {
    if (currentSize < 0) {
        System.out.println("Heap is empty.");
        return null;
    }
    Film minFilm = heap[0];
    heap[0] = heap[currentSize];
    currentSize--;
    minHeapify(i, 0);
    return minFilm;
}
private void minHeapify(int i) {
    int left = leftChild(i);
    int right = rightChild(i);
    int smallest = i;
    if (left <= currentSize && heap[left].popularityScore < heap[smallest].popularityScore) {
        smallest = left;
    }
    if (right <= currentSize && heap[right].popularityScore < heap[smallest].popularityScore) {
        smallest = right;
    }
}
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
Heap.java X ReviewLinkedList.java X FilmScreening.java X Actor.java X FilmLinkedList.java X FilmNode.java X ActorLinkedList.java X ActorNode.java X FilmMa
tory
minHeapify(i, 0);
return minFilm;
}
private void minHeapify(int i) {
    int left = leftChild(i);
    int right = rightChild(i);
    int smallest = i;
    if (left <= currentSize && heap[left].popularityScore < heap[smallest].popularityScore) {
        smallest = left;
    }
    if (right <= currentSize && heap[right].popularityScore < heap[smallest].popularityScore) {
        smallest = right;
    }
    if (smallest != i) {
        Film temp = heap[i];
        heap[i] = heap[smallest];
        heap[smallest] = temp;
        minHeapify(i, smallest);
    }
}
public void printHeap() {
    for (int i = 0; i <= currentSize; i++) {
        System.out.println(heap[i]);
    }
}
```

The minHeap Class

The RevenueNode class represents a single node in a data structure for managing revenue-related information. Each node stores attributes such as ticketPrice, viewership, and totalRevenue (calculated as $\text{ticketPrice} * \text{viewership}$). It also has a next pointer to link to the next node in the structure. The constructor initializes these attributes and sets the next pointer to null. This class is used to store and calculate revenue details efficiently in linked structures.

A screenshot of an Apache NetBeans IDE window titled 'filmManagement - Apache NetBeans IDE 19'. The 'RevenueNode.java' file is open, showing the following Java code:

```
package com.mycompany.filmmanagement;

public class RevenueNode {
    RevenueNode next;
    double ticketPrice;
    double viewership;
    double totalRevenue;

    public RevenueNode (double viewership , double ticketPrice) {

        this.ticketPrice = ticketPrice;
        this.viewership = viewership;
        this.totalRevenue = ticketPrice * viewership;
        next = null;
    }
}
```

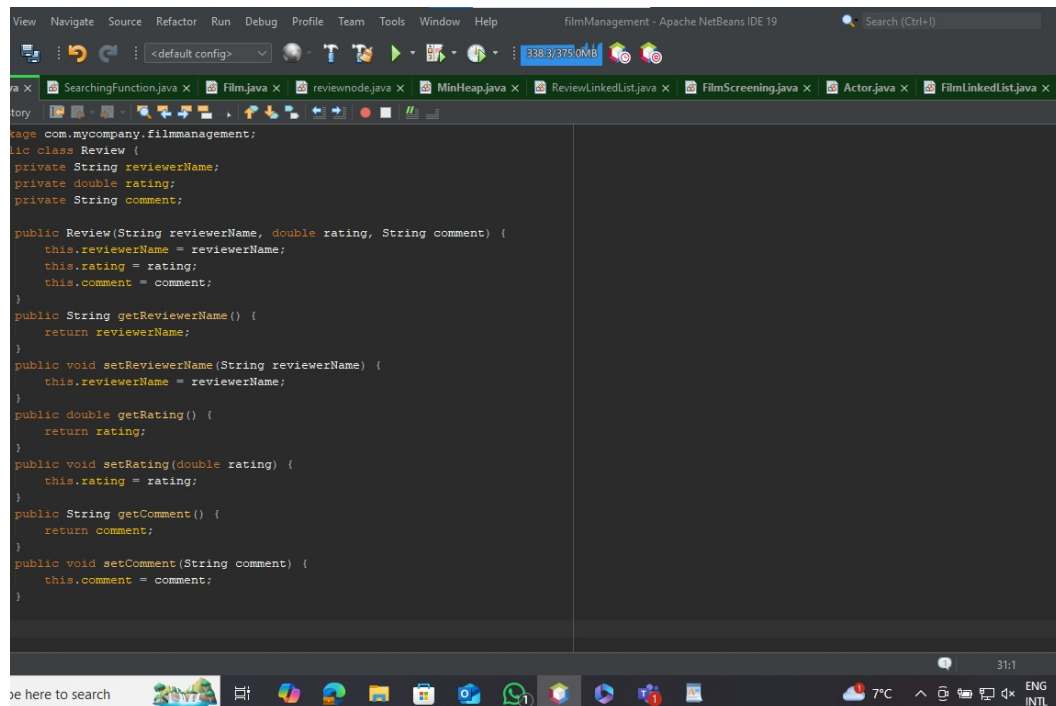
The IDE interface includes a menu bar (View, Navigate, Source, Refactor, Run, Debug, Profile, Team, Tools, Window, Help), a toolbar with icons for file operations and running, and a taskbar at the bottom with various application icons and system status indicators like temperature (6°C) and language (ENG INTL).

The RevenueNode Class

The Review class represents a review for a film, capturing key details such as the reviewer's name, the rating they provided, and their comments about the film. It consists of three main attributes: reviewerName, which stores the name of the reviewer; rating, a numerical value that reflects the reviewer's assessment of the film; and comment, a text field for any additional remarks or feedback.

The constructor initializes these attributes, ensuring that each review is created with all the necessary details. Additionally, the class provides getter and setter methods for each attribute, allowing easy access and modification of the review data while maintaining proper encapsulation.

This class plays an important role in a film management system by enabling the storage and management of individual reviews. It allows reviews to be created, updated, and retrieved efficiently, making it an essential component for organizing user feedback and providing insights about a film's reception.

A screenshot of the Apache NetBeans IDE interface. The top menu bar includes View, Navigate, Source, Refactor, Run, Debug, Profile, Team, Tools, Window, and Help. The toolbar shows various icons for file operations and running code. The main editor window displays the source code for the 'Review' class. The code is in Java and defines a class with private attributes for reviewerName, rating, and comment, along with their respective getters and setters. The package name is 'com.mycompany.filmmanagement'.

```
package com.mycompany.filmmanagement;

public class Review {
    private String reviewerName;
    private double rating;
    private String comment;

    public Review(String reviewerName, double rating, String comment) {
        this.reviewerName = reviewerName;
        this.rating = rating;
        this.comment = comment;
    }

    public String getReviewerName() {
        return reviewerName;
    }

    public void setReviewerName(String reviewerName) {
        this.reviewerName = reviewerName;
    }

    public double getRating() {
        return rating;
    }

    public void setRating(double rating) {
        this.rating = rating;
    }

    public String getComment() {
        return comment;
    }

    public void setComment(String comment) {
        this.comment = comment;
    }
}
```

The Review Class

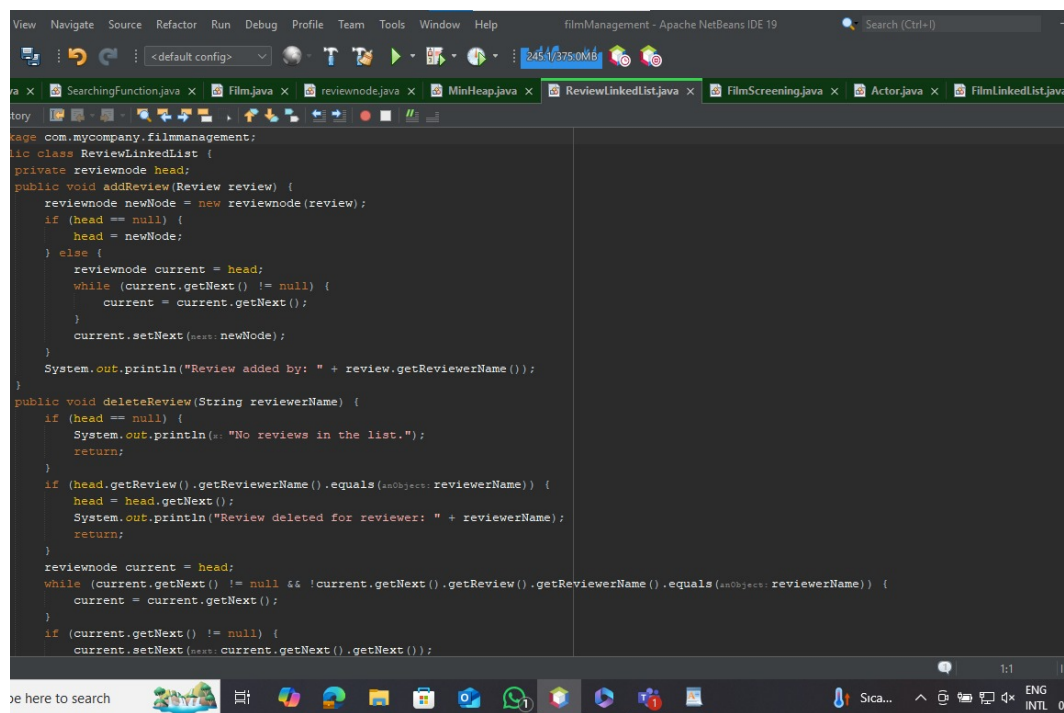
The *ReviewLinkedList* class manages a linked list of reviews, allowing operations such as adding, deleting, and printing reviews. The list is maintained using a head pointer, which references the first node in the list.

The `addReview(Review review)` method adds a new review to the end of the list. If the list is empty, the new review becomes the head. Otherwise, the method traverses the list to find the last node and appends the new review.

The `deleteReview(String reviewerName)` method removes a review based on the reviewer's name. If the list is empty, it prints a message indicating that there are no reviews. If the specified review is found, it is removed from the list, and an appropriate message is printed.

The `printReviews()` method displays all the reviews in the list. If the list is empty, it prints a message stating that there are no reviews. Otherwise, it traverses the list and prints each review's details, including the reviewer's name, rating, and comment.

This class provides a simple and efficient way to manage a dynamic list of film reviews.



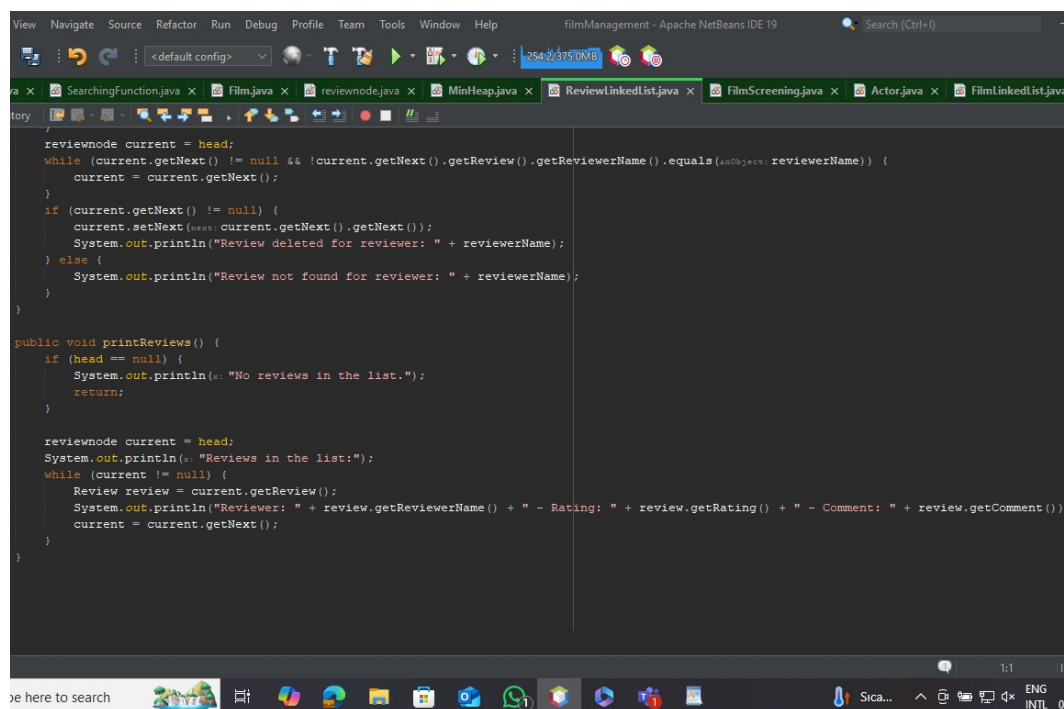
```
package com.mycompany.filmmanagement;

import java.util.Scanner;

public class ReviewLinkedList {
    private reviewnode head;

    public void addReview(Review review) {
        reviewnode newNode = new reviewnode(review);
        if (head == null) {
            head = newNode;
        } else {
            reviewnode current = head;
            while (current.getNext() != null) {
                current = current.getNext();
            }
            current.setNext(newNode);
        }
        System.out.println("Review added by: " + review.getReviewerName());
    }

    public void deleteReview(String reviewerName) {
        if (head == null) {
            System.out.println("No reviews in the list.");
            return;
        }
        if (head.getReview().getReviewerName().equals(reviewerName)) {
            head = head.getNext();
            System.out.println("Review deleted for reviewer: " + reviewerName);
            return;
        }
        reviewnode current = head;
        while (current.getNext() != null && !current.getNext().getReview().getReviewerName().equals(reviewerName)) {
            current = current.getNext();
        }
        if (current.getNext() != null) {
            current.setNext(current.getNext().getNext());
        }
    }
}
```

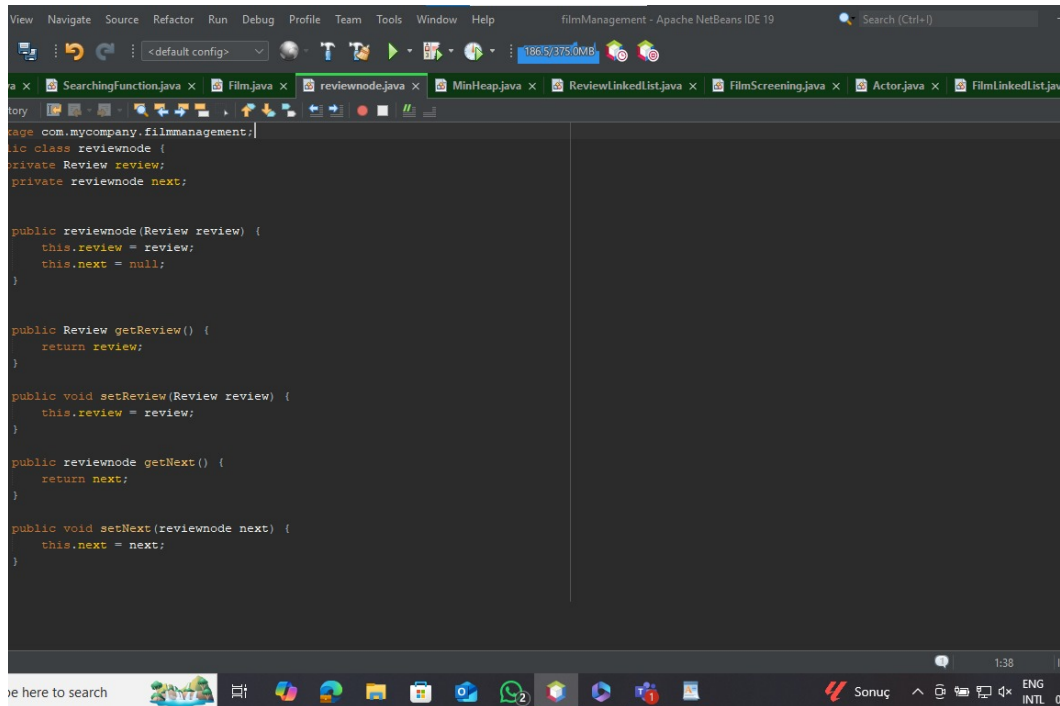


```
        reviewnode current = head;
        while (current.getNext() != null && !current.getNext().getReview().getReviewerName().equals(reviewerName)) {
            current = current.getNext();
        }
        if (current.getNext() != null) {
            current.setNext(current.getNext().getNext());
            System.out.println("Review deleted for reviewer: " + reviewerName);
        } else {
            System.out.println("Review not found for reviewer: " + reviewerName);
        }
    }

    public void printReviews() {
        if (head == null) {
            System.out.println("No reviews in the list.");
            return;
        }
        reviewnode current = head;
        System.out.println("Reviews in the list:");
        while (current != null) {
            Review review = current.getReview();
            System.out.println("Reviewer: " + review.getReviewerName() + " - Rating: " + review.getRating() + " - Comment: " + review.getComment());
            current = current.getNext();
        }
    }
}
```

The ReviewLinkedList Class

The reviewnode class represents a single node in a linked list of reviews. Each node contains two attributes: review, which stores a Review object, and next, which is a reference to the next reviewnode in the list. The constructor initializes the review and sets the next pointer to null. Getter and setter methods are provided to access and modify the review and next attributes. This class is essential for linking and managing multiple reviews in a linked list structure.



The reviewnode Class

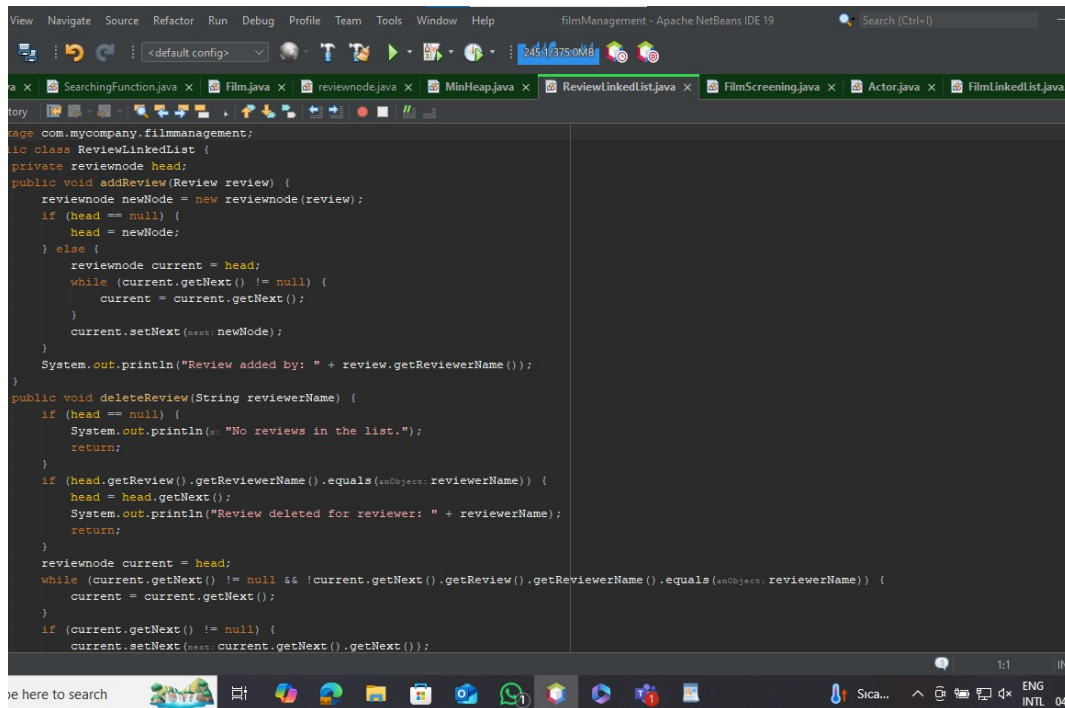
The SearchingFunction class implements a binary search tree (BST) to efficiently store and retrieve data. The class uses a root node as the entry point for the tree and provides methods to add, search, and print data stored in the tree.

The add(String name, Object data) method inserts a new node into the tree. It converts the key to lowercase and delegates the insertion to a recursive helper method, addRecursive, which places the new node in the correct position based on alphabetical order.

The searchRecursive(String name) method searches for a node by its key. It also uses a recursive helper function, which traverses the tree and returns the data of the matching node or null if no match is found.

The printAll() method prints all nodes in the tree in sorted order using the printInOrder helper function. This function performs an in-order traversal of the tree, ensuring that the nodes are printed in ascending order based on their keys.

This class provides an efficient way to manage and retrieve data, especially for applications requiring fast lookups and ordered traversal.

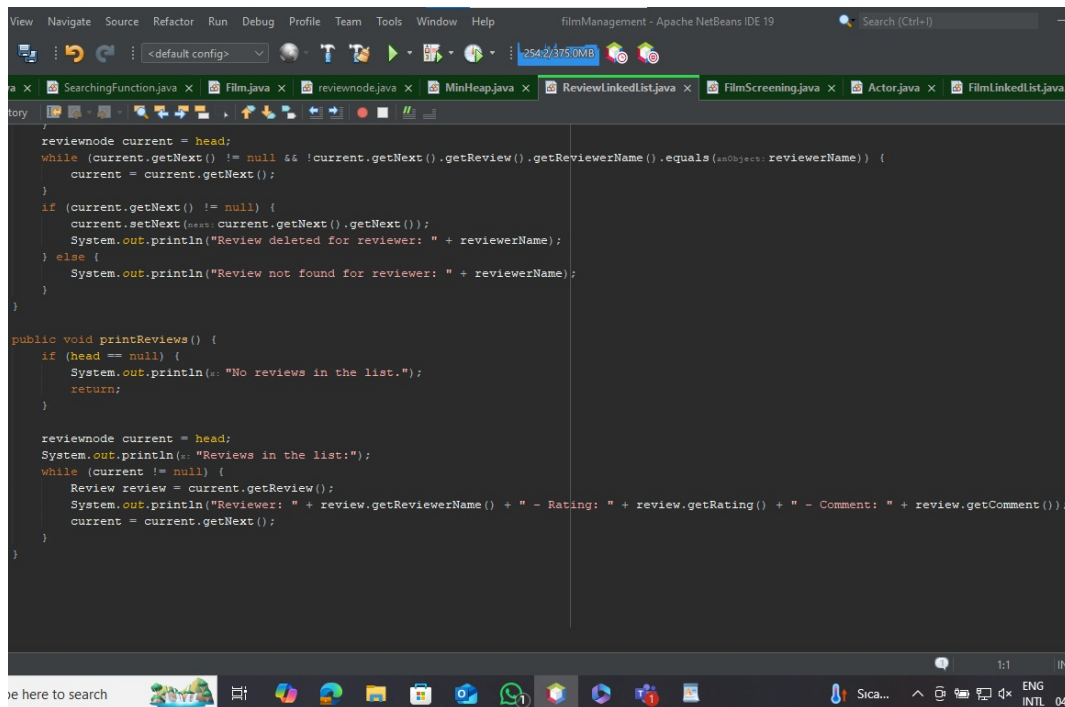


```
package com.mycompany.filmmanagement;

public class ReviewLinkedList {
    private reviewnode head;

    public void addReview(review review) {
        reviewnode newNode = new reviewnode(review);
        if (head == null) {
            head = newNode;
        } else {
            reviewnode current = head;
            while (current.getNext() != null) {
                current = current.getNext();
            }
            current.setNext(newNode);
        }
        System.out.println("Review added by: " + review.getReviewerName());
    }

    public void deleteReview(String reviewerName) {
        if (head == null) {
            System.out.println("No reviews in the list.");
            return;
        }
        if (head.getReview().getReviewerName().equals(reviewerName)) {
            head = head.getNext();
            System.out.println("Review deleted for reviewer: " + reviewerName);
            return;
        }
        reviewnode current = head;
        while (current.getNext() != null && !current.getNext().getReview().getReviewerName().equals(reviewerName)) {
            current = current.getNext();
        }
        if (current.getNext() != null) {
            current.setNext(current.getNext().getNext());
        }
    }
}
```



```
reviewnode current = head;
while (current.getNext() != null && !current.getNext().getReview().getReviewerName().equals(reviewerName)) {
    current = current.getNext();
}
if (current.getNext() != null) {
    current.setNext(current.getNext().getNext());
    System.out.println("Review deleted for reviewer: " + reviewerName);
} else {
    System.out.println("Review not found for reviewer: " + reviewerName);
}

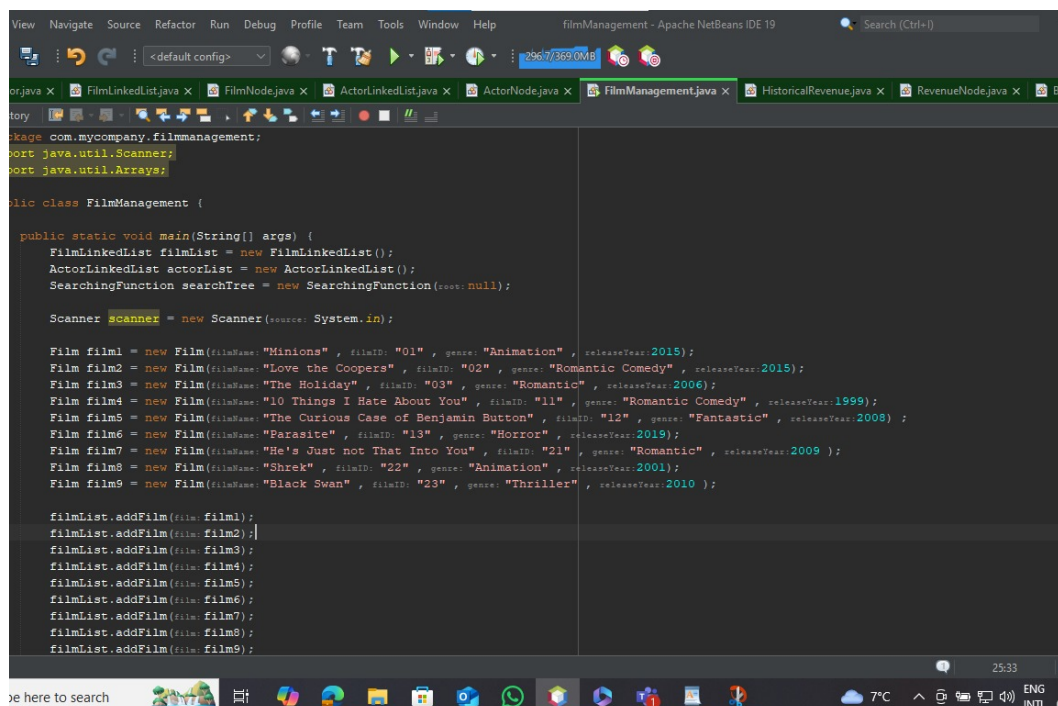
public void printReviews() {
    if (head == null) {
        System.out.println("No reviews in the list.");
        return;
    }
    reviewnode current = head;
    System.out.println("Reviews in the list:");
    while (current != null) {
        Review review = current.getReview();
        System.out.println("Reviewer: " + review.getReviewerName() + " - Rating: " + review.getRating() + " - Comment: " + review.getComment());
        current = current.getNext();
    }
}
```

The SearchingFunction Class

Now that we have created and detailed the individual classes, the next step is to bring everything together in the main method. The main method will serve as the central point where all these classes interact to form a functional film management system. Here, we will create objects of the various classes, link them together, and demonstrate their functionality through practical use cases.

The FilmManagement class serves as the main entry point of the program, integrating various functionalities to manage films, actors, and related data. It utilizes a FilmLinkedList to store and manage films, an ActorLinkedList for actor data, and a SearchingFunction implemented as a binary search tree to efficiently search for films and actors. The program begins by creating sample films and actors, which are added to their respective data structures for demonstration purposes. Through a user-friendly menu in the main method, users can perform a variety of actions, including adding new films and actors, updating film details, searching for specific films or actors, and simulating film revenue based on viewership and ticket prices. Additionally, users can view revenue history, display films ranked by popularity, and provide reviews with feedback for films.

Each menu option is designed to interact seamlessly with the underlying data structures, ensuring smooth functionality. For example, films and actors can be quickly searched using the binary search tree, while reviews and other details are managed through linked lists. The program also handles edge cases, such as invalid inputs or missing data, by providing appropriate error messages. With its modular design, the FilmManagement class brings together all individual components into a cohesive system, demonstrating the power of object-oriented programming and efficient data handling techniques.



```
package com.mycompany.filmmanagement;
import java.util.Scanner;
import java.util.Arrays;

public class FilmManagement {

    public static void main(String[] args) {
        FilmLinkedList filmList = new FilmLinkedList();
        ActorLinkedList actorList = new ActorLinkedList();
        SearchingFunction searchTree = new SearchingFunction(null);

        Scanner scanner = new Scanner(System.in);

        Film film1 = new Film(filmName: "Minions", filmID: "01", genre: "Animation", releaseYear: 2015);
        Film film2 = new Film(filmName: "Love the Coopers", filmID: "02", genre: "Romantic Comedy", releaseYear: 2015);
        Film film3 = new Film(filmName: "The Holiday", filmID: "03", genre: "Romantic", releaseYear: 2006);
        Film film4 = new Film(filmName: "10 Things I Hate About You", filmID: "11", genre: "Romantic Comedy", releaseYear: 1999);
        Film film5 = new Film(filmName: "The Curious Case of Benjamin Button", filmID: "12", genre: "Fantastic", releaseYear: 2008);
        Film film6 = new Film(filmName: "Parasite", filmID: "13", genre: "Horror", releaseYear: 2019);
        Film film7 = new Film(filmName: "He's Just Not That Into You", filmID: "21", genre: "Romantic", releaseYear: 2009);
        Film film8 = new Film(filmName: "Shrek", filmID: "22", genre: "Animation", releaseYear: 2001);
        Film film9 = new Film(filmName: "Black Swan", filmID: "23", genre: "Thriller", releaseYear: 2010);

        filmList.addFilm(film: film1);
        filmList.addFilm(film: film2);
        filmList.addFilm(film: film3);
        filmList.addFilm(film: film4);
        filmList.addFilm(film: film5);
        filmList.addFilm(film: film6);
        filmList.addFilm(film: film7);
        filmList.addFilm(film: film8);
        filmList.addFilm(film: film9);
    }
}
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl-I)
259.3/369.0MB

prj.java x FilmLinkedList.java x FilmNode.java x ActorLinkedList.java x ActorNode.java x FilmManagement.java x HistoricalRevenue.java x RevenueNode.java x BS
tory

film1.addImdb(imdb: 6.4 , fileName: "Minions");
film2.addImdb(imdb: 5.8 , fileName: "Love the Coopers");
film3.addImdb(imdb: 7.0 , fileName: "The Holiday");
film4.addImdb(imdb: 7.3 , fileName: "10 Things I Hate About You");
film5.addImdb(imdb: 7.8 , fileName: "The Curious Case of Benjamin Button");
film6.addImdb(imdb: 8.5 , fileName: "Parasite");
film7.addImdb(imdb: 6.5 , fileName: "He's Just Not That Into You");
film8.addImdb(imdb: 7.9 , fileName: "Shrek");
film9.addImdb(imdb: 8.0 , fileName: "Black Swan");

Actor actor1 = new Actor(actorName: "Steve Martin" , actorID: "A1");
Actor actor2 = new Actor(actorName: "Diane Keaton" , actorID: "A2");
Actor actor3 = new Actor(actorName: "Kate Winslet" , actorID: "A3");
Actor actor4 = new Actor(actorName: "Cameron Diaz" , actorID: "A4");
Actor actor5 = new Actor(actorName: "Heath Ledger" , actorID: "A5");
Actor actor6 = new Actor(actorName: "Julia Stiles" , actorID: "A6");
Actor actor7 = new Actor(actorName: "Brad Pitt" , actorID: "A7");
Actor actor8 = new Actor(actorName: "Cate Blanchett" , actorID: "A8");
Actor actor9 = new Actor(actorName: "Song Kang-ho" , actorID: "A9");
Actor actor10 = new Actor(actorName: "Lee Sun-kyun" , actorID: "A10");
Actor actor11 = new Actor(actorName: "Cho Yeo-jeong" , actorID: "A11");
Actor actor12 = new Actor(actorName: "Ginnifer Goodwin" , actorID: "A12");
Actor actor13 = new Actor(actorName: "Jennifer Aniston" , actorID: "A13");
Actor actor14 = new Actor(actorName: "Jennifer Connelly" , actorID: "A14");
Actor actor15 = new Actor(actorName: "Natalie Portman" , actorID: "A15");
Actor actor16 = new Actor(actorName: "Mila Kunis" , actorID: "A16");

actorList.addActor(actor: actor1);
actorList.addActor(actor: actor2);
actorList.addActor(actor: actor3);
actorList.addActor(actor: actor4);
actorList.addActor(actor: actor5);
actorList.addActor(actor: actor6);
actorList.addActor(actor: actor7);
actorList.addActor(actor: actor8);
actorList.addActor(actor: actor9);
actorList.addActor(actor: actor10);
actorList.addActor(actor: actor11);
actorList.addActor(actor: actor12);
actorList.addActor(actor: actor13);
actorList.addActor(actor: actor14);
actorList.addActor(actor: actor15);
actorList.addActor(actor: actor16);

searchTree.add(name: film1.getFilmName() , data: film1);
searchTree.add(name: film2.getFilmName() , data: film2);
searchTree.add(name: film3.getFilmName() , data: film3);
searchTree.add(name: film4.getFilmName() , data: film4);
searchTree.add(name: film5.getFilmName() , data: film5);
searchTree.add(name: film6.getFilmName() , data: film6);

boolean running = true;
while (running) {
    System.out.println("\nSelect an option:");
    System.out.println("1. Add Film");
    System.out.println("2. Update Film");
    System.out.println("3. Add Actor");
}
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl-I)
184.9/369.0MB

prj.java x FilmLinkedList.java x FilmNode.java x ActorLinkedList.java x ActorNode.java x FilmManagement.java x HistoricalRevenue.java x RevenueNode.java x BS
tory

actorList.addActor(actor: actor1);
actorList.addActor(actor: actor2);
actorList.addActor(actor: actor3);
actorList.addActor(actor: actor4);
actorList.addActor(actor: actor5);
actorList.addActor(actor: actor6);
actorList.addActor(actor: actor7);
actorList.addActor(actor: actor8);
actorList.addActor(actor: actor9);
actorList.addActor(actor: actor10);
actorList.addActor(actor: actor11);
actorList.addActor(actor: actor12);
actorList.addActor(actor: actor13);
actorList.addActor(actor: actor14);
actorList.addActor(actor: actor15);
actorList.addActor(actor: actor16);

searchTree.add(name: actor5.getActorName() , data: actor5);
searchTree.add(name: actor6.getActorName() , data: actor6);
searchTree.add(name: actor7.getActorName() , data: actor7);
searchTree.add(name: actor8.getActorName() , data: actor8);
searchTree.add(name: actor9.getActorName() , data: actor9);
searchTree.add(name: actor10.getActorName() , data: actor10);
searchTree.add(name: actor11.getActorName() , data: actor11);
searchTree.add(name: actor12.getActorName() , data: actor12);
searchTree.add(name: actor13.getActorName() , data: actor13);
searchTree.add(name: actor14.getActorName() , data: actor14);
searchTree.add(name: actor15.getActorName() , data: actor15);
searchTree.add(name: actor16.getActorName() , data: actor16);

film1.addRevenue(viewership: 200000 , ticketPrice: 10.0);
film2.addRevenue(viewership: 150000 , ticketPrice: 8.5);
film3.addRevenue(viewership: 300000 , ticketPrice: 7.5);
film4.addRevenue(viewership: 180000 , ticketPrice: 4.0);
film5.addRevenue(viewership: 400000 , ticketPrice: 12.0);
film6.addRevenue(viewership: 500000 , ticketPrice: 16.0);
film7.addRevenue(viewership: 220000 , ticketPrice: 9.0);
film8.addRevenue(viewership: 350000 , ticketPrice: 10.0);
film9.addRevenue(viewership: 250000 , ticketPrice: 11.0);

boolean running = true;
while (running) {
    System.out.println("\nSelect an option:");
    System.out.println("1. Add Film");
    System.out.println("2. Update Film");
    System.out.println("3. Add Actor");
}
```



```
View  Navigate  Source  Refactor  Run  Debug  Profile  Team  Tools  Window  Help  filmManagement - Apache NetBeans IDE 19  Search (Ctrl+I)
<default config>  363.2/369.0MB
prj.java X  FilmLinkedList.java X  FilmNode.java X  ActorLinkedList.java X  ActorNode.java X  FilmManagement.java X  HistoricalRevenue.java X  RevenueNode.java X  BS
tory  [Icons]

boolean running = true;
while (running) {
    System.out.println("\nSelect an option:");
    System.out.println("1. Add Film");
    System.out.println("2. Update Film");
    System.out.println("3. Add Actor");
    System.out.println("4. Search Film/Actor");
    System.out.println("5. Simulate Revenue");
    System.out.println("6. Retrieve Revenue History");
    System.out.println("7. Display Popularity Rankings");
    System.out.println("8. Viewer Feedback");
    System.out.println("9. Exit");
    System.out.print("Your choice: ");
    int choice = scanner.nextInt();
    scanner.nextLine();

    switch (choice) {
        case 1:
            System.out.print("Enter film name: ");
            String fileName = scanner.nextLine();
            System.out.print("Enter film ID: ");
            String filmID = scanner.nextLine();
            System.out.print("Enter genre: ");
            String genre = scanner.nextLine();
            System.out.print("Enter release year: ");
            int releaseYear = scanner.nextInt();
            Film newFilm = new Film(fileName, filmID, genre, releaseYear);
            filmList.addFilm(film: newFilm);
            searchTree.add(name: fileName, data: newFilm);
            System.out.println("Film added successfully.");
            filmList.printFilms();
            break;
```

```
View  Navigate  Source  Refactor  Run  Debug  Profile  Team  Tools  Window  Help  filmManagement - Apache NetBeans IDE 19  Search (Ctrl+I)
<default config>  301.2/369.0MB
prj.java X  FilmLinkedList.java X  FilmNode.java X  ActorLinkedList.java X  ActorNode.java X  FilmManagement.java X  HistoricalRevenue.java X  RevenueNode.java X  BS
tory  [Icons]

        break;

        case 2:
            System.out.print("Enter film name to update: ");
            String filmToUpdate = scanner.nextLine();
            Film filmToEdit = (Film) searchTree.searchRecursive(name: filmToUpdate);
            if (filmToEdit != null) {
                System.out.print("Enter new genre: ");
                String newGenre = scanner.nextLine();
                filmToEdit.setGenre(genre: newGenre);
                System.out.println("Film updated successfully.");
                System.out.println("Updated Film: " + filmToEdit.toString());
            } else {
                System.out.println("Film not found.");
            }
            break;

        case 3:
            System.out.println("\nActor List:");
            actorList.printActors();
            System.out.print("Enter actor name: ");
            String actorName = scanner.nextLine();
            System.out.print("Enter actor ID: ");
            String actorID = scanner.nextLine();
            Actor newActor = new Actor(actorName, actorID);
            actorList.addActor(actor: newActor);
            searchTree.add(name: actorName, data: newActor);
            System.out.println("Actor added successfully.");
            actorList.printActors();
            break;

        case 4:
```

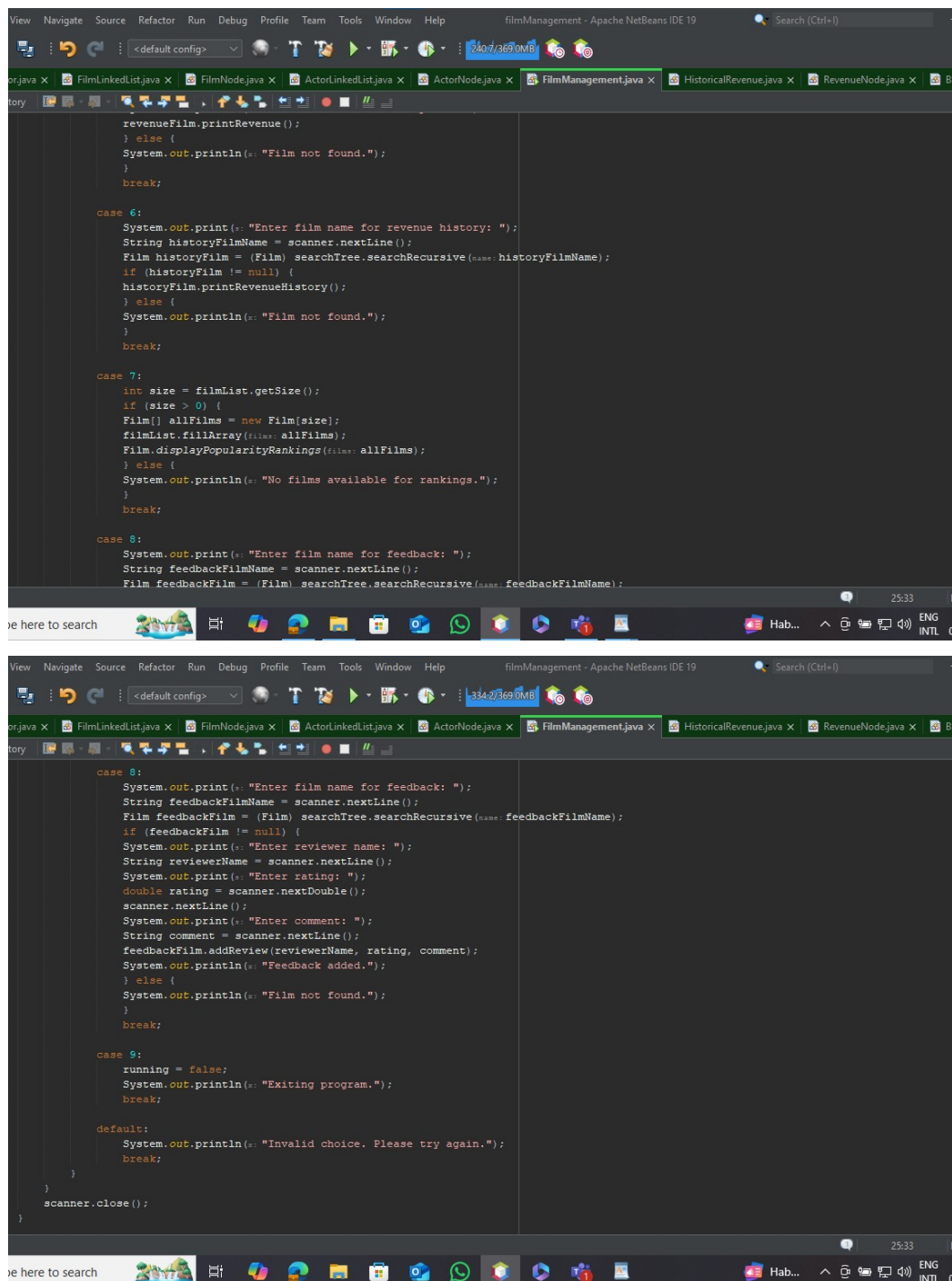
```
View  Navigate  Source  Refactor  Run  Debug  Profile  Team  Tools  Window  Help  filmManagement - Apache NetBeans IDE 19  Search (Ctrl+I)
<default config>  304.2/369.0MB
prj.java X  FilmLinkedList.java X  FilmNode.java X  ActorLinkedList.java X  ActorNode.java X  FilmManagement.java X  HistoricalRevenue.java X  RevenueNode.java X  BS
tory  [Icons]

        actorList.printActors();
        break;

        case 4:
            System.out.print("Enter film or actor name to search: ");
            String searchName = scanner.nextLine();
            Object result = searchTree.searchRecursive(name: searchName);
            if (result != null) {
                System.out.println("Found: " + result);
            } else {
                System.out.println("Not found.");
            }
            break;

        case 5:
            System.out.print("Enter film name for revenue simulation: ");
            String revenueFilmName = scanner.nextLine();
            Film revenueFilm = (Film) searchTree.searchRecursive(name: revenueFilmName);
            if (revenueFilm != null) {
                System.out.print("Enter viewership count: ");
                double viewership = scanner.nextDouble();
                System.out.print("Enter ticket price: ");
                double ticketPrice = scanner.nextDouble();
                revenueFilm.addRevenue(viewership, ticketPrice);
                System.out.println("Revenue simulated and updated.");
                revenueFilm.printRevenue();
            } else {
                System.out.println("Film not found.");
            }
            break;

        case 6:
```



The Main Method

The output of the **FilmManagement** program showcases the system's functionality by reflecting the results of various user actions, such as adding films or actors, updating film details, searching, and viewing revenue or reviews. It provides clear and structured feedback for each operation, displaying messages for successful actions, search results, and any errors like missing films or invalid inputs. This output highlights the seamless interaction between the program's components, ensuring a smooth and intuitive user experience.


```
View  Navigate  Source  Refactor  Run  Debug  Profile  Team  Tools  Window  Help  filmManagement - Apache NetBeans IDE 19  Search (Ctrl+I)
<default config> 242.7/422.0MB
Source  History  package com.mycompany.filmmanagement;
import java.util.Scanner;
import java.util.Arrays;
4
Management)
-----[ jar ]-----
sources:3.3.1:resources (default-resources) @ filmManagement ---
on existing resourceDirectory C:\Users\Zehra\OneDrive\Documents\NetBeansProjects\filmManagement\src\main\resources
mpiler:3.11.0:compile (default-compile) @ filmManagement ---
s detected - recompiling the module! :source
ing 17 source files with javac [debug target 21] to target\classes
ec:3.1.0:exec (default-cli) @ filmManagement ---
an option:
Film
ate Film
Actor
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
hoice:
```

```
View  Navigate  Source  Refactor  Run  Debug  Profile  Team  Tools  Window  Help  filmManagement - Apache NetBeans IDE 19  Search (Ctrl+I)
<default config> 197.3/422.0MB
Source  History  package com.mycompany.filmmanagement;
import java.util.Scanner;
import java.util.Arrays;
4
Management)
play Popularity Rankings
wer Feedback
t
hoice: 1
film name: Deadpool 2
film ID: 31
genre: Action/Comedy
release year: 2018
dded successfully.
s
he Coopers
liday
ngs I Hate About You
rious Case of Benjamin Button
te
ust not That Into You
Swan
ol 2
an option:
Film
```

```
View  Navigate  Source  Refactor  Run  Debug  Profile  Team  Tools  Window  Help  filmManagement - Apache NetBeans IDE 19  Search (Ctrl+I)
<default config> 300.4/422.0MB
Source  History  package com.mycompany.filmmanagement;
import java.util.Scanner;
import java.util.Arrays;
4
Management)
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
hoice: 2
film name to update: Minions
new genre: Comedy
pdated successfully.
d Film: Film Name: Minions, ID: 01, Genre: Comedy, Release Year: 2015
an option:
Film
ate Film
Actor
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
hoice:
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
240/8/422.0MB
...va FilmLinkedList.java X FilmNode.java X ActorLinkedList.java X ActorNode.java X FilmManagement.java X HistoricalRevenue.java
Source History
1 package com.myccompany.filmmanagement;
2 import java.util.Scanner;
3 import java.util.Arrays;
4
Management)
t
hoice: 3
List:
Name: Steve Martin, Actor ID: A1
Name: Diane Keaton, Actor ID: A2
Name: Kate Winlest, Actor ID: A3
Name: Cameron Diaz, Actor ID: A4
Name: Heath Ledger, Actor ID: A5
Name: Julia Stiles, Actor ID: A6
Name: Brad Pitt, Actor ID: A7
Name: Cate Blanchett, Actor ID: A8
Name: Song Kang-ho, Actor ID: A9
Name: Lee Sun-kyun, Actor ID: A10
Name: Cho Yeo-jeong, Actor ID: A11
Name: Ginnifer Goodwin, Actor ID: A12
Name: Jennifer Aniston, Actor ID: A13
Name: Jennifer Connelly, Actor ID: A14
Name: Natalie Portman, Actor ID: A15
Name: Mila Kunis, Actor ID: A16
actor name: Ryan Reynolds
actor ID: A17
Run (filmManagement) 260:10
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
473/3/422.0MB
...va FilmLinkedList.java X FilmNode.java X ActorLinkedList.java X ActorNode.java X FilmManagement.java X HistoricalRevenue.java
Source History
1 package com.myccompany.filmmanagement;
2 import java.util.Scanner;
3 import java.util.Arrays;
4
Management)
Name: Mila Kunis, Actor ID: A16
actor name: Ryan Reynolds
actor ID: A17
added successfully.
Name: Steve Martin, Actor ID: A1
Name: Diane Keaton, Actor ID: A2
Name: Kate Winlest, Actor ID: A3
Name: Cameron Diaz, Actor ID: A4
Name: Heath Ledger, Actor ID: A5
Name: Julia Stiles, Actor ID: A6
Name: Brad Pitt, Actor ID: A7
Name: Cate Blanchett, Actor ID: A8
Name: Song Kang-ho, Actor ID: A9
Name: Lee Sun-kyun, Actor ID: A10
Name: Cho Yeo-jeong, Actor ID: A11
Name: Ginnifer Goodwin, Actor ID: A12
Name: Jennifer Aniston, Actor ID: A13
Name: Jennifer Connelly, Actor ID: A14
Name: Natalie Portman, Actor ID: A15
Name: Mila Kunis, Actor ID: A16
Name: Ryan Reynolds, Actor ID: A17
Run (filmManagement) 260:10
```

```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19 Search (Ctrl+I)
280/3/422.0MB
...va FilmLinkedList.java X FilmNode.java X ActorLinkedList.java X ActorNode.java X FilmManagement.java X HistoricalRevenue.java
Source History
1 package com.myccompany.filmmanagement;
2 import java.util.Scanner;
3 import java.util.Arrays;
4
Management)
Actor
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
hoice: 4
film or actor name to search: Heath Ledger
com.myccompany.filmmanagement.Actor@2e817b38
an option:
Film
ate Film
Actor
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
hoice: |
Run (filmManagement) 260:10
```

```
View  Navigate  Source  Refactor  Run  Debug  Profile  Team  Tools  Window  Help  filmManagement - Apache NetBeans IDE 19  Search (Ctrl+I)
<default config> 275.6/422.0MB
Source  History
1 package com.mycompany.filmmanagement;
2 import java.util.Scanner;
3
4
Management)
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
choice: 6
film name for revenue history: Parasite
e History for Parasite (ID: 13):
ship: 500000.0, Ticket Price: 16.0, Total Revenue: 8000000.0

an option:
Film
ate Film
Actor
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
choice:

Run (filmManagement) x 260:10
```

```
View  Navigate  Source  Refactor  Run  Debug  Profile  Team  Tools  Window  Help  filmManagement - Apache NetBeans IDE 19  Search (Ctrl+I)
<default config> 275.6/422.0MB
Source  History
1 package com.mycompany.filmmanagement;
2 import java.util.Scanner;
3
4
Management)
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
choice: 6
film name for revenue history: Parasite
e History for Parasite (ID: 13):
ship: 500000.0, Ticket Price: 16.0, Total Revenue: 8000000.0

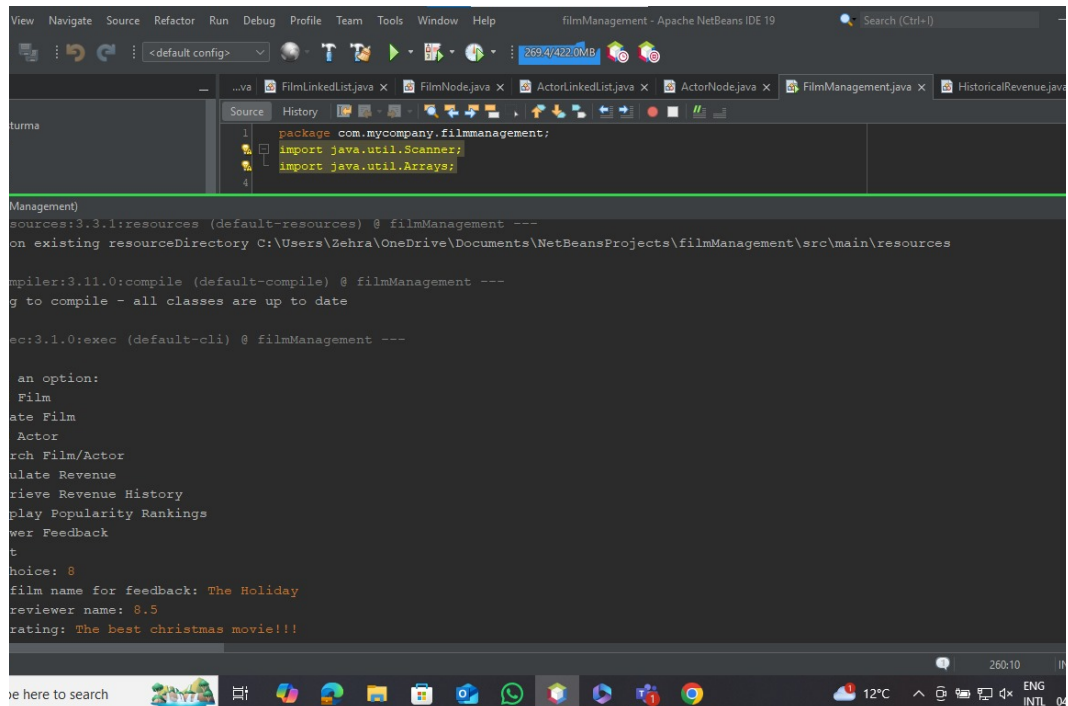
an option:
Film
ate Film
Actor
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
choice:

Run (filmManagement) x 260:10
```

```
View  Navigate  Source  Refactor  Run  Debug  Profile  Team  Tools  Window  Help  filmManagement - Apache NetBeans IDE 19  Search (Ctrl+I)
<default config> 164.7/422.0MB
Source  History
1 package com.mycompany.filmmanagement;
2 import java.util.Scanner;
3
4
Management)
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
choice: 7
ying Popularity Rankings:
te - Popularity Score: 85.0
Swan - Popularity Score: 80.0
- Popularity Score: 79.0
rious Case of Benjamin Button - Popularity Score: 78.0
ngs I Hate About You - Popularity Score: 73.0
liday - Popularity Score: 70.0
ust not That Into You - Popularity Score: 65.0
s - Popularity Score: 64.0
he Coopers - Popularity Score: 59.0
ol 2 - Popularity Score: 0.0

an option:
Film
ate Film
Actor
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
choice:

Run (filmManagement) x 260:10
```



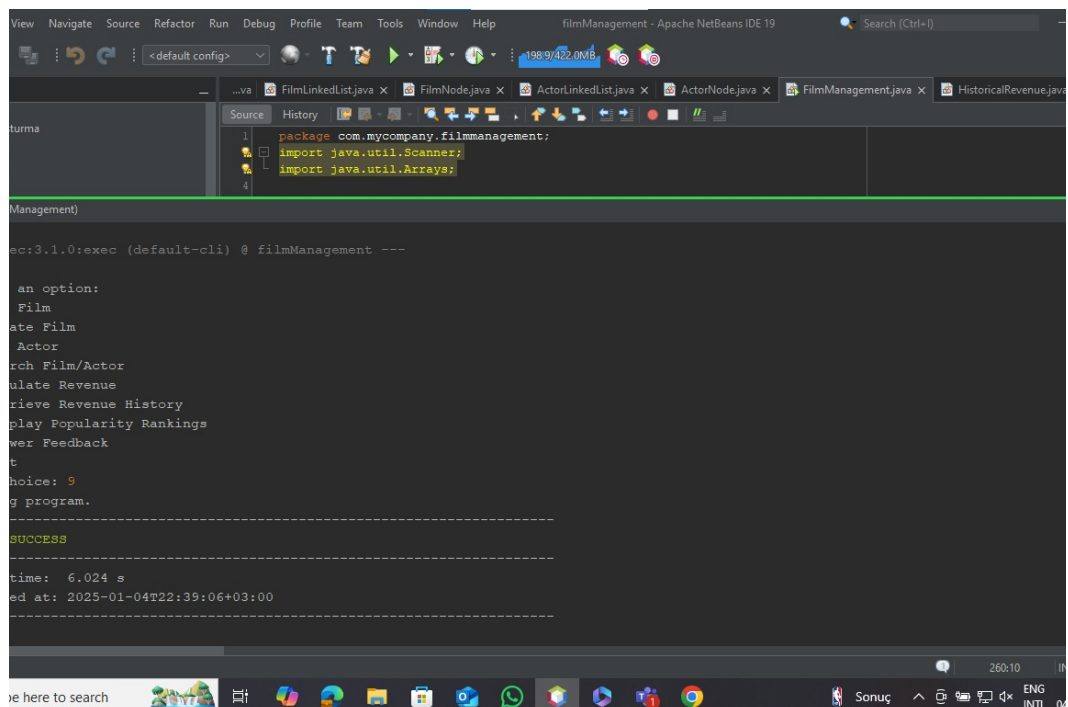
```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19
...Va FilmLinkedList.java X FilmNode.java X ActorLinkedList.java X ActorNode.java X FilmManagement.java X HistorcalRevenue.java
Source History
package com.mycompany.filmmanagement;
import java.util.Scanner;
import java.util.Arrays;

Management)
sources:3.3.1:resources (default-resources) @ filmManagement ---
on existing resourceDirectory C:\Users\Zehra\OneDrive\Documents\NetBeansProjects\filmManagement\src\main\resources

mpiler:3.11.0:compile (default-compile) @ filmManagement ---
g to compile - all classes are up to date

ec:3.1.0:exec (default-cli) @ filmManagement ---

an option:
Film
ate Film
Actor
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
hoice: 8
film name for feedback: The Holiday
reviewer name: 8.5
rating: The best christmas movie!!!
```



```
View Navigate Source Refactor Run Debug Profile Team Tools Window Help filmManagement - Apache NetBeans IDE 19
...Va FilmLinkedList.java X FilmNode.java X ActorLinkedList.java X ActorNode.java X FilmManagement.java X HistorcalRevenue.java
Source History
package com.mycompany.filmmanagement;
import java.util.Scanner;
import java.util.Arrays;

Management)
ec:3.1.0:exec (default-cli) @ filmManagement ---

an option:
Film
ate Film
Actor
rch Film/Actor
ulate Revenue
rieve Revenue History
play Popularity Rankings
wer Feedback
t
hoice: 9
g program.

SUCCESS

time: 6.024 s
ed at: 2025-01-04T22:39:06+03:00
```

The outputs

Thus, we have reached the end of the **FilmManagement** application. This program offers a comprehensive solution for managing films, actors, revenues, and reviews, ensuring seamless functionality and effective integration.

